Artificial Intelligence

SEARCH

Search: Planning Before Acting

Search is a way for an AI agent to think through possible actions before choosing what to do.

Instead of acting randomly, the agent asks:

“What actions can I take, what states can they lead to, and which path reaches my goal?”

Step 01
Current Situation

Step 02
Possible Actions

Step 03
Future States

Goal
Chosen Action

Examples



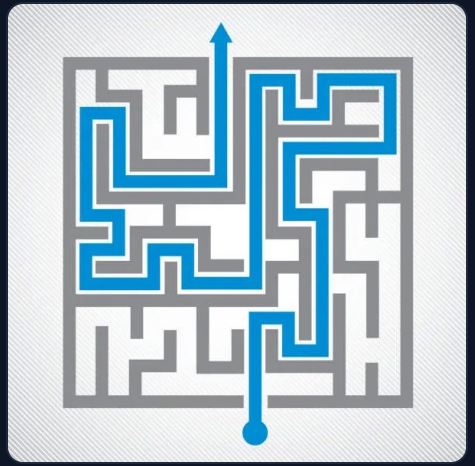
GPS Navigation

Route Finding & Mapping



Puzzle Solving

State Optimization Games



Maze Traversal

Pathfinding Algorithms

Search as Rational Action

A rational agent chooses actions that help it achieve its goal. In search problems, rational behavior means:

- **Understanding** the current state
- **Knowing** the possible actions
- **Predicting** the result of each action
- **Choosing** a path that reaches the goal
- **Preferring** the best or lowest-cost path

Tic-Tac-Toe Example

A rational X player does not place X randomly. It checks possible moves and chooses a move that improves the chance of winning or avoids losing.



What Is a Search Problem?

The Core Question

“Starting from where we are now, what sequence of actions gets us to a desired goal?”

The Expected Output

Usually a **solution**, which is a defined sequence of actions that successfully reaches the goal.

Five Core Components Needed

Initial State The starting point of the agent

Actions All possible moves available to the agent

Transition Model What happens as a result of each action

Goal Test Condition to check if the goal is reached

Path Cost Numerical cost assigned to a sequence of actions



The Search Problem Components

State
A description of the current situation

Initial state
Where the agent starts

Actions
Choices available from a state

Transition model
What happens after an action

Goal test
Checks whether the goal is reached

Path cost
Measures how expensive a path is

Tic-Tac-Toe as a Search Problem

Agent

The player making a decision

State

Current board configuration

Initial State

Empty board, or the board position where search begins

Actions

Place a mark in any empty cell

Transition Model

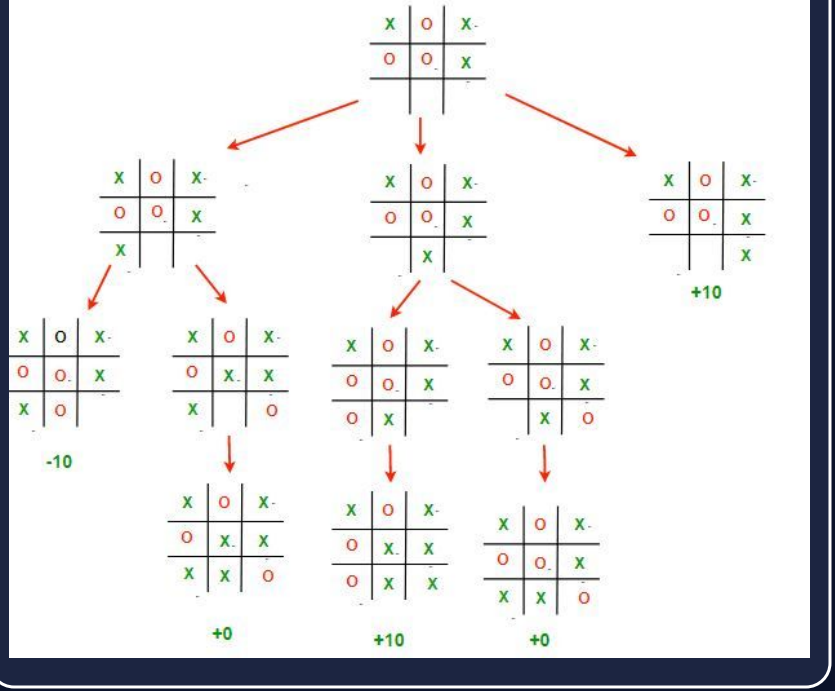
Resulting board after an action

Goal Test

X wins, O wins, or no empty cells remain

Utility

Win = +10, Draw = 0, Loss = −10



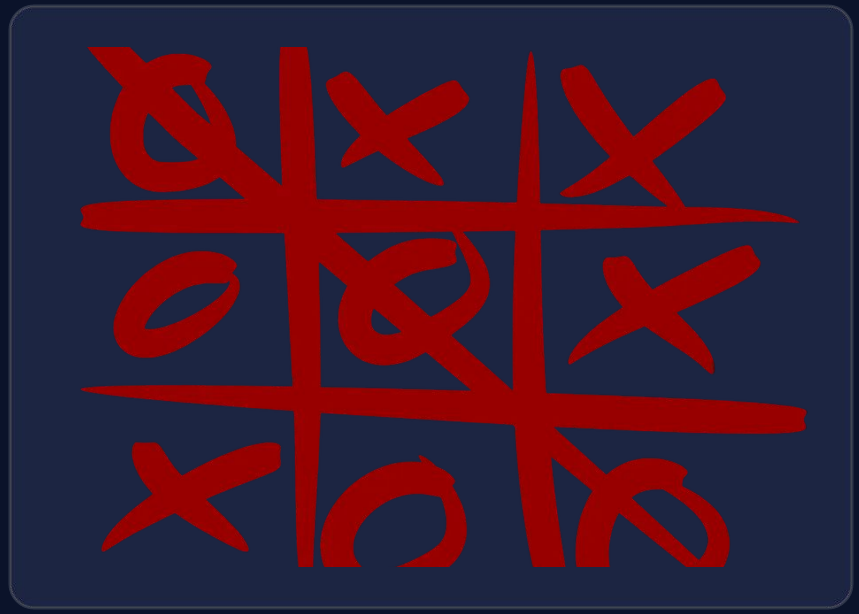
State Representation

A state should contain all information needed to make the next decision.

9-Cell Content
The content/mark inside each of the 9 cells.

Active Turn
Whose turn it currently is to make a move.

Terminal Check
Whether the game has ended: X wins, O wins, or draw.



The state is a snapshot of the game at one moment. It tells us what is in every cell and whose turn is next.

Why Representation Matters

The way we define the state changes the size and difficulty of the problem.



Too Little Info

State = only X locations

Problem: The AI does not know where O has played, so it cannot know legal moves or detect threats.



Too Much Info

State = board + player names + colors + previous animations

Problem: The AI wastes memory and computation on details that do not affect the decision.



Good Representation

State = board config + player turn

Solution: This is enough to determine legal moves and evaluate the game effectively.

State Space

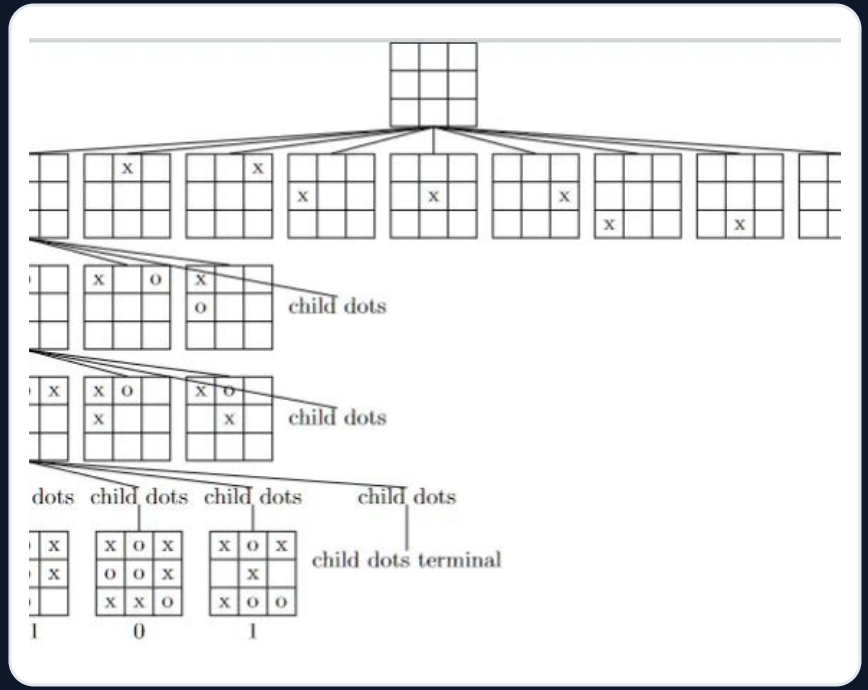
The set of all states reachable from the initial state by legal actions. The current state + all states that can be reached after 1 move, 2 moves, 3 moves, and so on.

Tic-Tac-Toe Game Tree

- **Start State:** Empty 3×3 board
- **Branching:** X has 9 initial moves
- **Turns:** O has 8 responses, then X has 7...
- **Terminal:** Game ends at win, lose, or draw

Graph Formulation

- **Nodes:** Represent distinct board configurations
- **Edges:** Represent legal transitions (moves)



State-Space Complexity

Even simple problems can have many possible states.

Possible Boards

Each cell can be:

Empty • X • O

$3^9 = 19{,}683$

But Not All Are Legal

X and O take turns

X starts

Game stops after a win

Legal State Space

Reachable board states

under Tic-Tac-Toe rules

5,478

As the number of possible states grows, search becomes harder.

For Tic-Tac-Toe, which state representation is best?

Evaluating the optimal data required for an AI agent to make decisions.

Multiple Choice Options

A. Current board only

B. Current board + whose turn it is

C. Current board + whose turn it is + board color + player names

D. Only the number of empty cells

Correct Answer

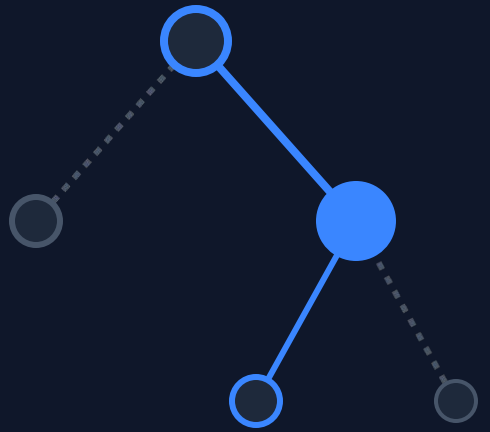
Option B

Why this is best

The AI needs **the board state** and the **current player's turn** to accurately determine legal actions and future states.

Once the AI has the state space,

How does it decide which states to explore first?



Problem Solving & Search

Defining a "Solution"

CONCEPT 01

Standard Solution

A sequence of actions that leads from the initial state to the goal state.

CONCEPT 02

Optimal Solution

A solution that has the lowest path cost among all solutions.

Primary Objective

Search algorithms' goal is to find the optimal solution, or at least a solution that is close to optimal.

Uninformed Search

Search Strategy & The Frontier

The Big Idea

The agent does not know where the goal is, so it must explore possible states systematically.

The Frontier

States discovered but not yet explored.

Represents the agent's current set of possible next choices during search.

Key Algorithms

- Depth-First Search (DFS)
- Breadth-First Search (BFS)
- Uniform-Cost Search (UCS)

General Search Structure

Keep frontier

Choose node

Expand

Add nodes

The Core Decision

Which node should we explore next?

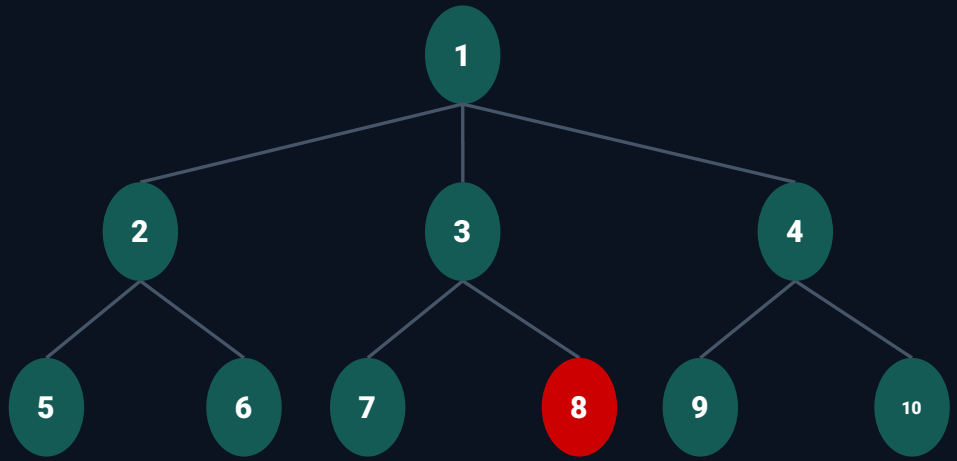
Example

Assumptions

- Start node = 1
- Goal node = 8
- Children added left to right

The Goal

We will trace how DFS and BFS behave differently on this identical tree structure.



Depth-First Search (DFS)

Search Strategy

Go Deep First

Explores as deep as possible along each branch before backtracking to try other directions.

Mechanism

Stack-Based (LIFO)

Uses a **Last In, First Out** stack. The newest node added to the frontier is always explored first.

Intuition & Analogy

Exploring a Maze

DFS acts like exploring a single hallway all the way to the very end before returning to try any other hallways.

DFS Traversal Example

EXECUTION TRACE

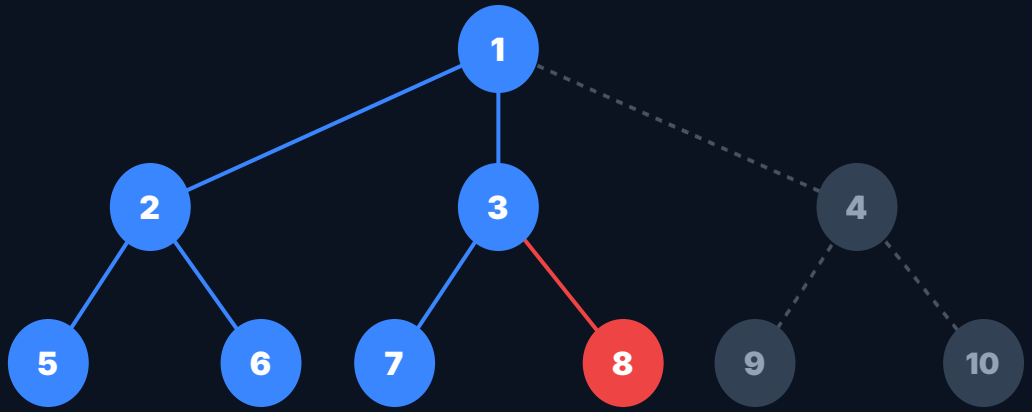
DFS Exploration

If DFS explores the left branch first, it travels deep before backtracking.

TRAVERSAL ORDER

1 → 2 → 5 → 6 → 3 → 7 → 8

✓ Goal found at Node 8



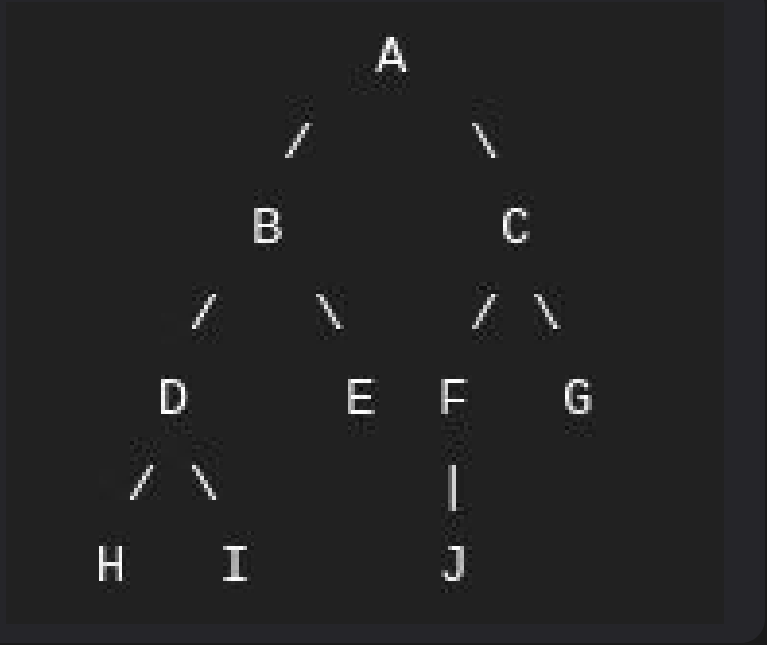
Traverse This Tree Using DFS

DFS RULES

Traversing the Tree

Explore children **from left to right**. The target goal node is **E**.

Starting at **A**, what is the traversal order?



Traverse This Tree Using DFS

DFS RULES

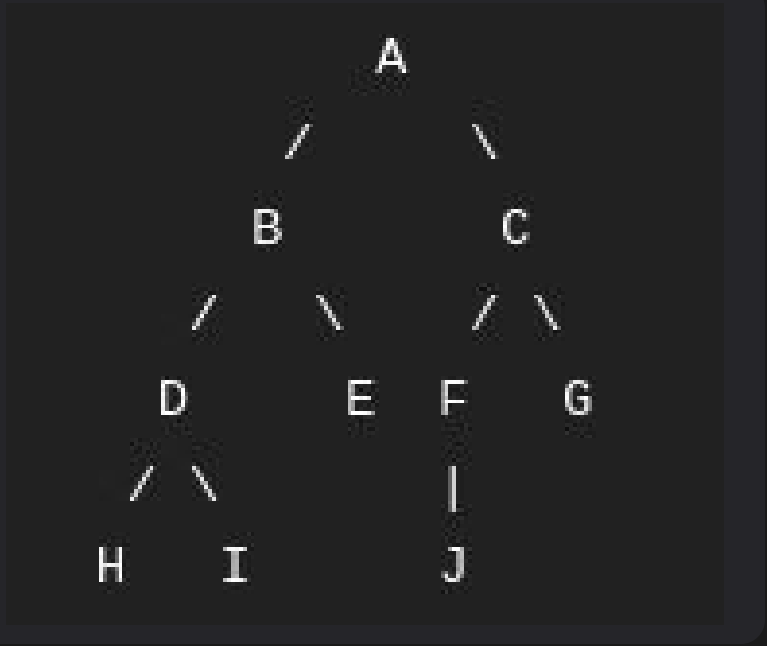
Traversing the Tree

Explore children **from left to right**. The target goal node is **E**.

Starting at **A**, what is the traversal order?

CORRECT TRAVERSAL ORDER

A → B → D → H → I → E



Traverse This Tree Using DFS

DFS QUESTION

Suppose the goal is G. Which nodes will DFS visit before finding G?

A. A, B, C, G

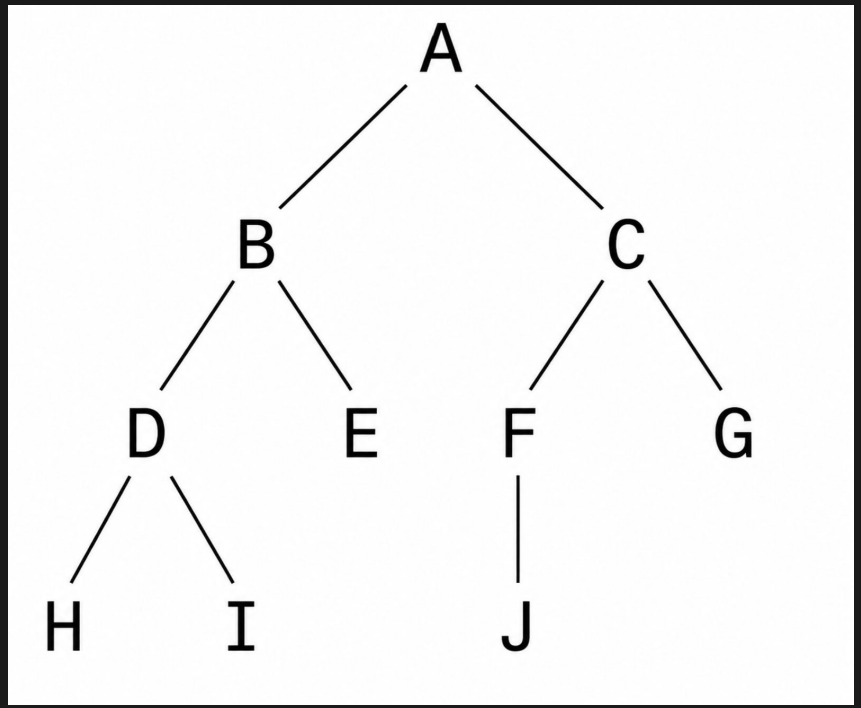
B. A, C, F, G

C. A, B, D. H, I, E, C, F, J, G

D. A, B, D, E, C, F, J, G

Correct Answer: C

A → B → D → H → I → E → C → F → J → G



Traverse This Graph Using DFS

DFS Problem Setup

Suppose the goal is D. Which nodes will DFS visit before finding D?

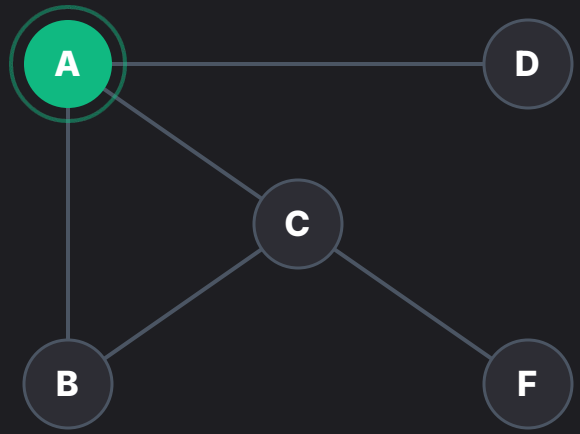
Rule: Explore children alphabetically when possible.

Visited

A				

Stack

B	C	D		



Traverse This Graph Using DFS

DFS Problem Setup

Suppose the goal is D. Which nodes will DFS visit before finding D?

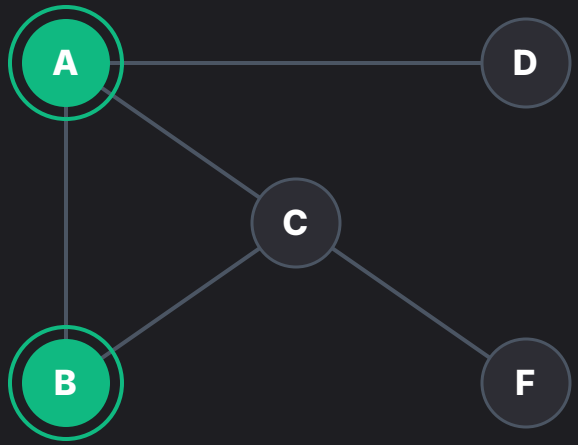
Rule: Explore children alphabetically when possible.

Visited

A	B			

Stack

C	D			



Traverse This Graph Using DFS

DFS Problem Setup

Suppose the goal is D. Which nodes will DFS visit before finding D?

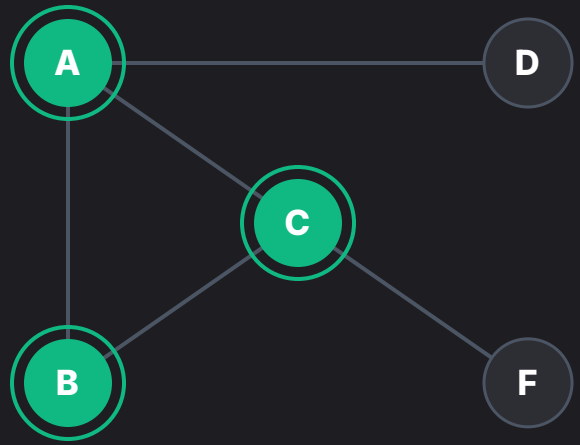
Rule: Explore children alphabetically when possible.

Visited

A	B	C		

Stack

F	D			



Traverse This Graph Using DFS

DFS Problem Setup

Suppose the goal is D. Which nodes will DFS visit before finding D?

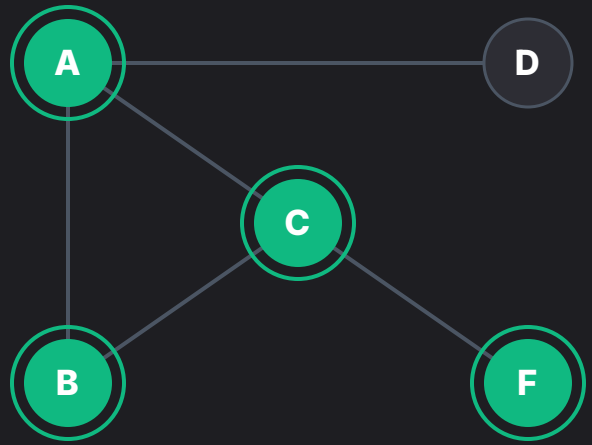
Rule: Explore children alphabetically when possible.

Visited

A	B	C	F	

Stack

D				



This is a good example because it demonstrates the main DFS idea: **D is directly connected to A, but DFS does not necessarily choose the shortest route. It goes deep along the first available branch before backtracking.**

Traverse This Graph Using DFS

DFS Problem Setup

Suppose the goal is D. Which nodes will DFS visit before finding D?

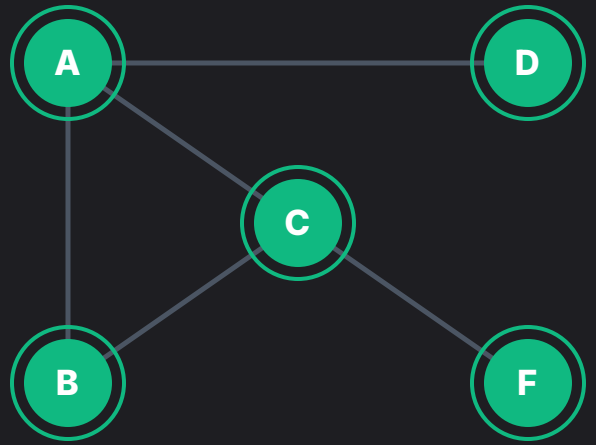
Rule: Explore children alphabetically when possible.

Visited

A	B	C	F	D

Stack

Why did we add the visited set?
The search can loop forever because it keeps rediscovering states.



Traverse This Graph Using DFS

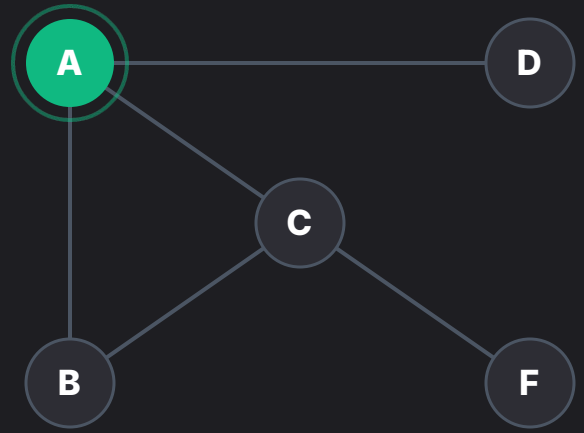
DFS Problem Setup

Suppose the goal is D. Which nodes will DFS visit before finding D?

Rule: Explore children alphabetically when possible.

Visited





Traverse This Graph Using DFS

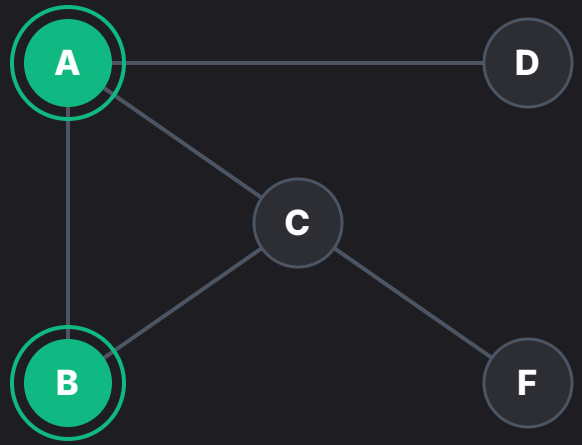
DFS Problem Setup

Suppose the goal is D. Which nodes will DFS visit before finding D?

Rule: Explore children alphabetically when possible.

Visited





Traverse This Graph Using DFS

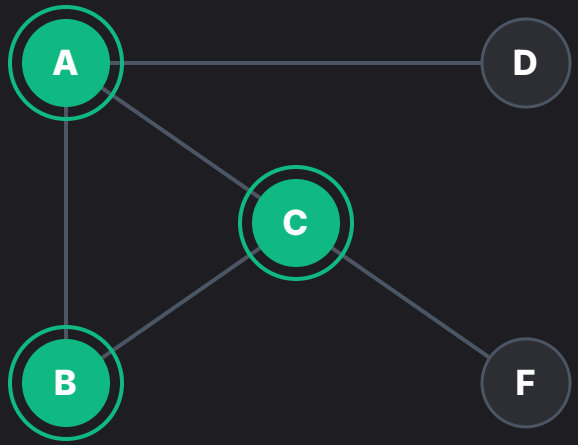
DFS Problem Setup

Suppose the goal is D. Which nodes will DFS visit before finding D?

Rule: Explore children alphabetically when possible.

Visited





Traverse This Graph Using DFS

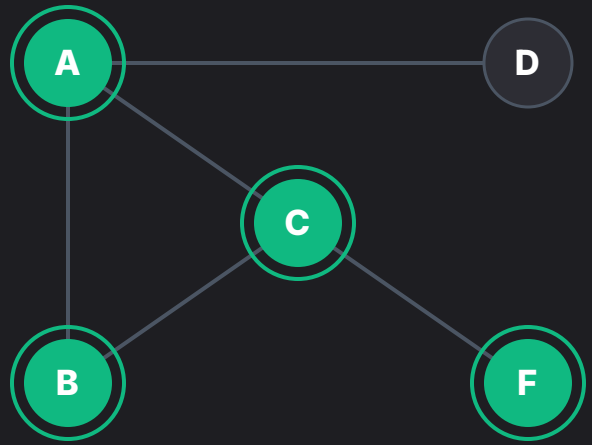
DFS Problem Setup

Suppose the goal is D. Which nodes will DFS visit before finding D?

Rule: Explore children alphabetically when possible.

Visited





This is a good example because it demonstrates the main DFS idea: **D is directly connected to A, but DFS does not necessarily choose the shortest route. It goes deep along the first available branch before backtracking.**

Traverse This Graph Using DFS

DFS Problem Setup

Suppose the goal is D. Which nodes will DFS visit before finding D?

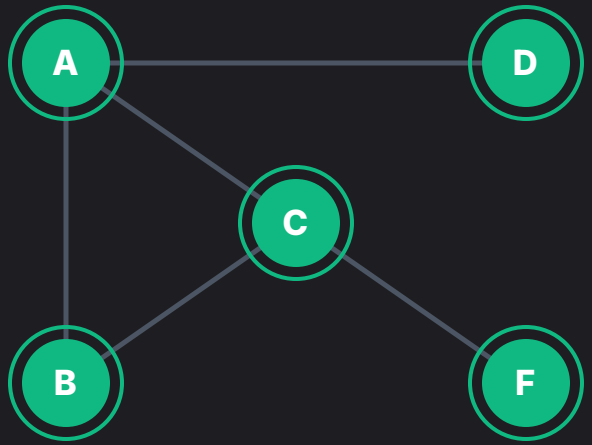
Rule: Explore children alphabetically when possible.

Visited

A B C F D

Why did we add the visited set?

The search can loop forever because it keeps rediscovering states.



Pac-Man: DFS Pathfinding

Question & Setup

If Pac-Man uses DFS, will it necessarily find the shortest route to G?

P = Pac-Man

G = Goal

▮ = Wall

Answer: No.

DFS may follow one corridor deeply even when a shorter route exists elsewhere.

DFS goes deep until it reaches a dead end or wall, then it backtracks and tries another path.

Grid Environment Map



DFS: Strengths & Weaknesses

Strengths

- Simple to implement
- Uses less memory than BFS in many cases
- Can be fast if the solution is deep along the first explored path

Weaknesses

- Not guaranteed to find the shortest path (optimal solution)
- May be incomplete in spaces with cycles
- Can get stuck in very deep paths or cycles
- May explore a bad direction for a long time

DFS: Node Expansion Quiz

QUESTION

In DFS, which node is expanded first?

A. The node closest to the goal

B. The node with the lowest path cost

C. The most recently added node in the frontier

CORRECT ANSWER

Option C

DFS uses a Stack

Last In, First Out. The most recent added node is expanded first.

Breadth-First Search

CORE CONCEPT

Breadth-First Search explores level by level.

It uses a queue: **First In, First Out**

The oldest node in the frontier is explored first.

ANALOGY

Analogy:

BFS checks all places one step away, then all places two steps away, then all places three steps away.

BFS Traversal Example

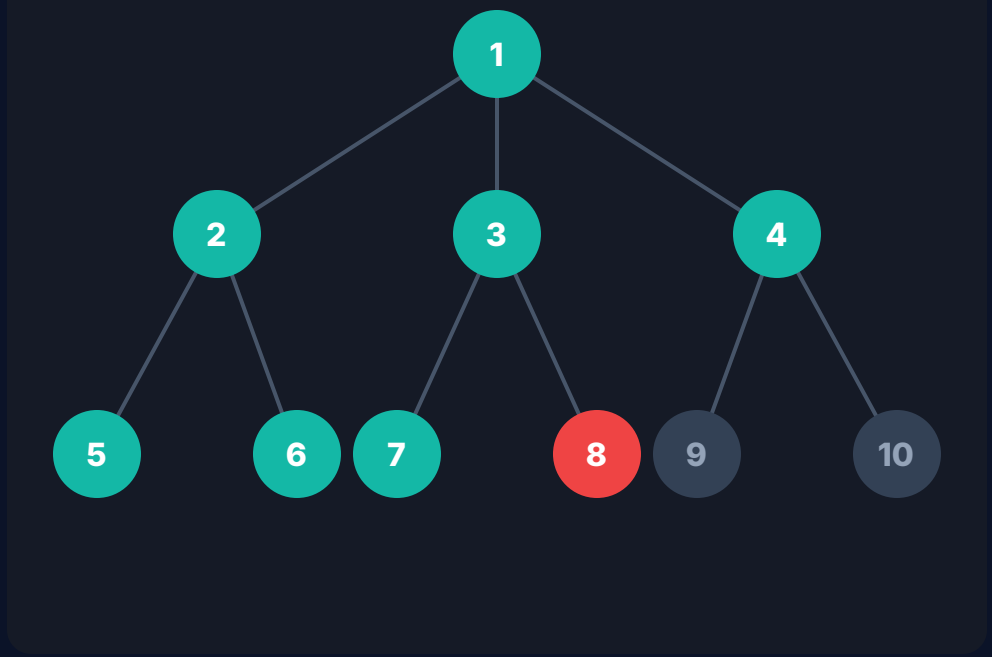
Traversal Order

If BFS explores left first:

BFS Order:

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8

Goal node 8 is successfully found!



BFS Traversal Example

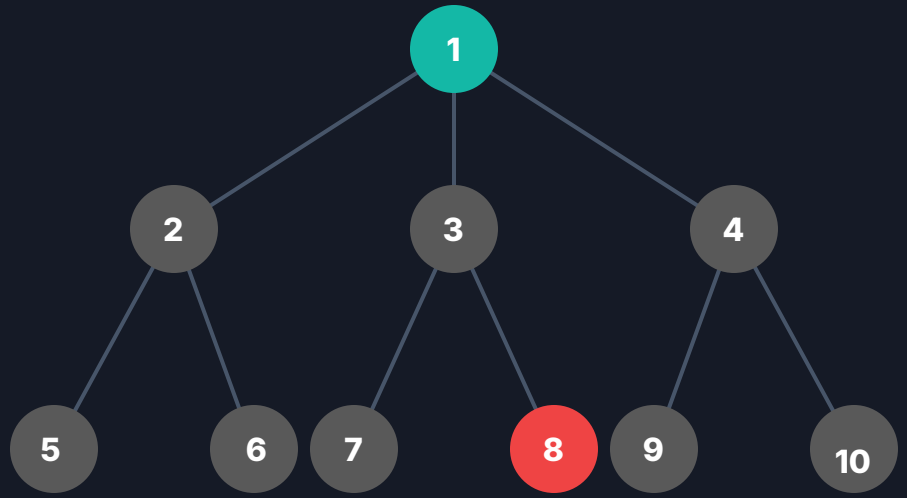
Traversal Order

If BFS explores left first:

BFS Order:

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8

Goal node 8 is successfully found!



BFS Traversal Example

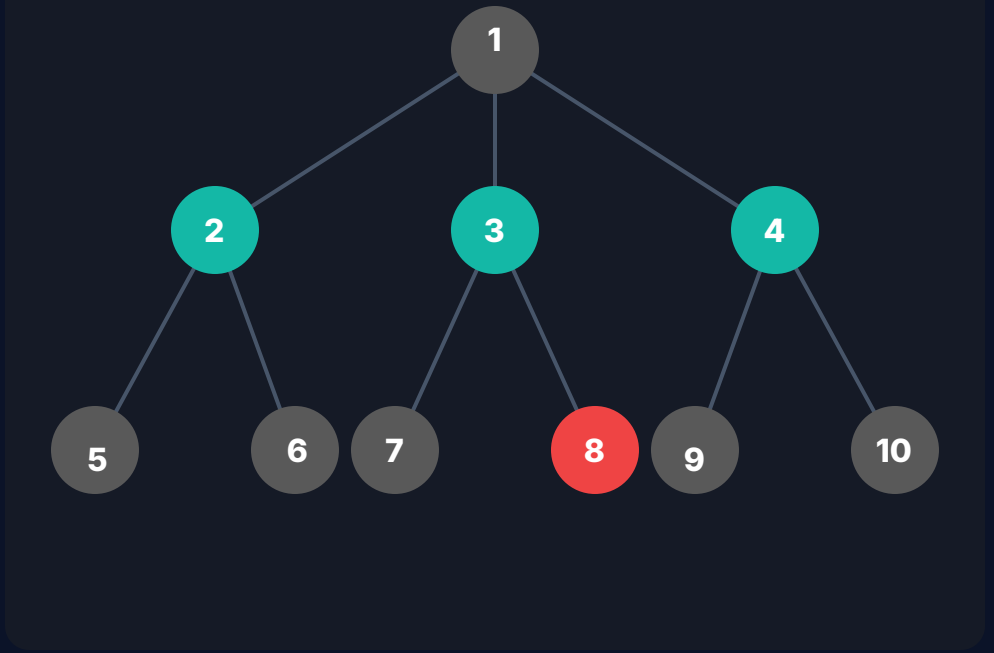
Traversal Order

If BFS explores left first:

BFS Order:

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8

Goal node 8 is successfully found!



BFS Traversal Example

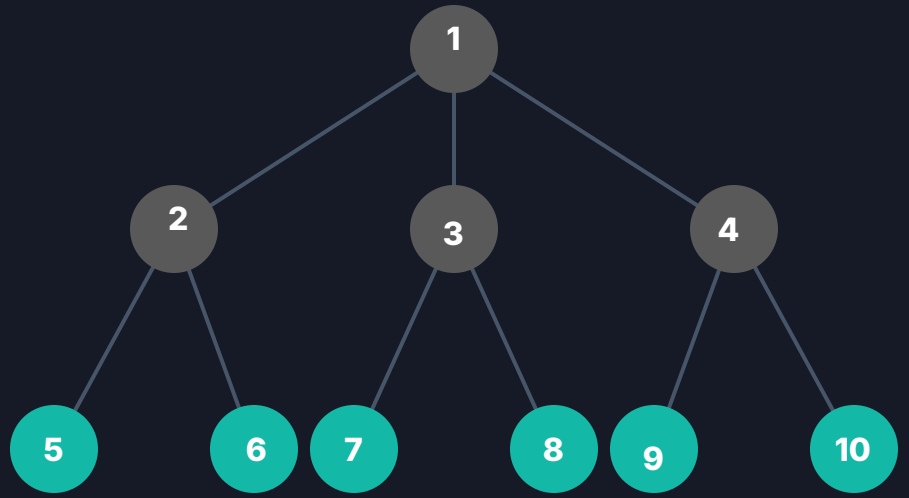
Traversal Order

If BFS explores left first:

BFS Order:

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8

Goal node 8 is successfully found!



Traverse This Tree Using BFS

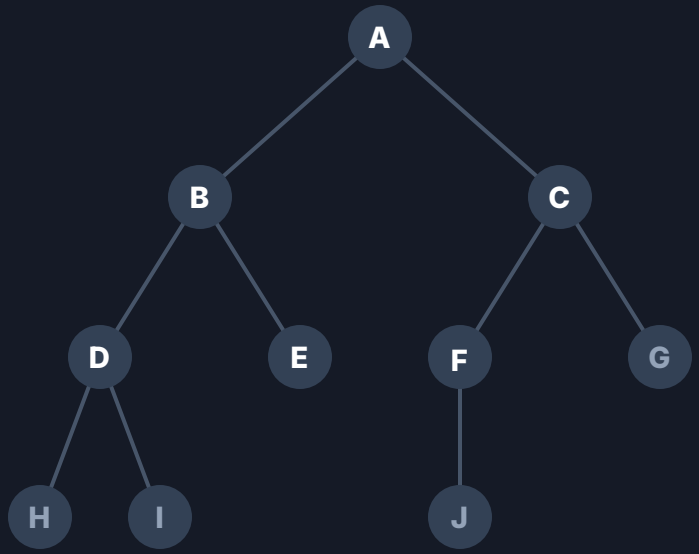
TRAVERSAL RULES

Assume we always explore children from left to right, and the goal node is F.

Starting at A, what is the BFS traversal order?

Answer

A → B → C → D → E → F



Traverse This Tree Using DFS

Suppose the goal is J. Which nodes will DFS visit before finding J?

A. A, B, C, F, J

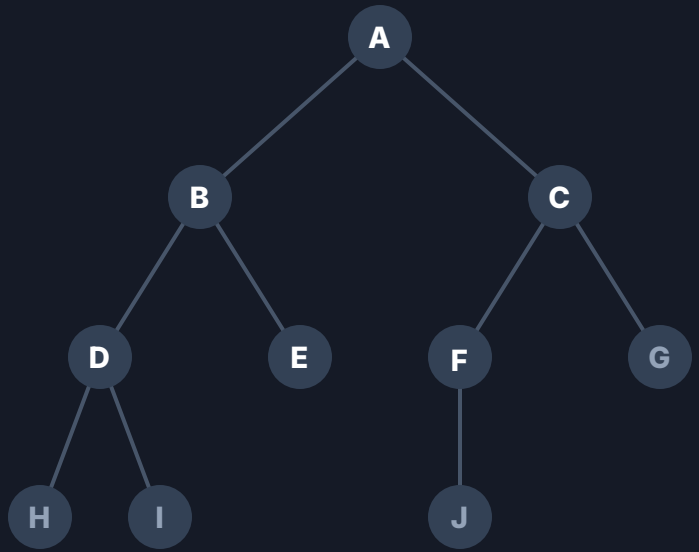
B. A, B, C, D, E, F, G, H, I, J

C. A, B, D, H, I, J

D. A, B, D, H, I, E, C, F, J

CORRECT ANSWER: D

A → B → D → H → I → E → C → F → J



Traverse This Graph Using BFS

BFS Problem Setup

Suppose the goal is D. Which nodes will BFS visit before finding D?

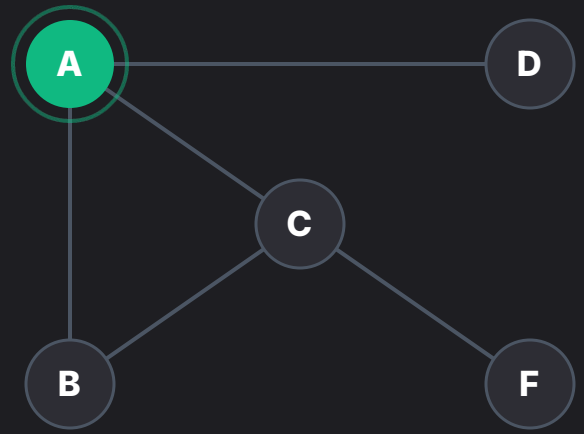
Rule: Explore children alphabetically when possible.

Visited

A				

Stack

B	C	D		



Traverse This Graph Using BFS

BFS Problem Setup

Suppose the goal is D. Which nodes will BFS visit before finding D?

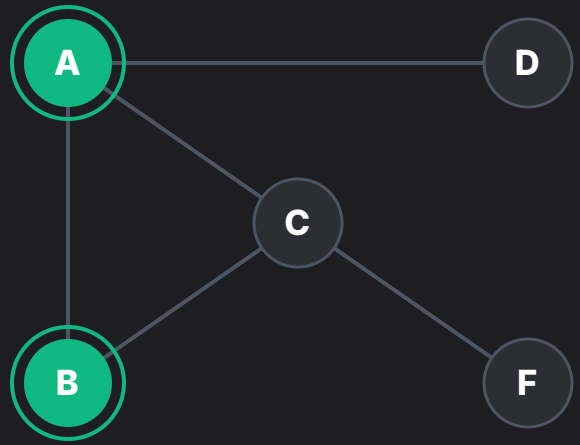
Rule: Explore children alphabetically when possible.

Visited

A	B			

Stack

C	D			



Traverse This Graph Using BFS

BFS Problem Setup

Suppose the goal is D. Which nodes will BFS visit before finding D?

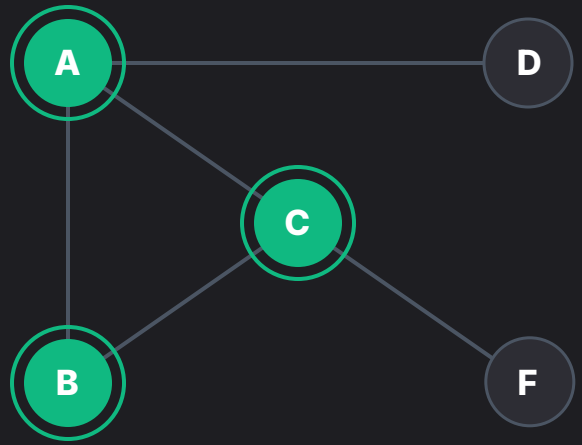
Rule: Explore children alphabetically when possible.

Visited

A	B	C		

Stack

D				



Traverse This Graph Using BFS

BFS Problem Setup

Suppose the goal is D. Which nodes will BFS visit before finding D?

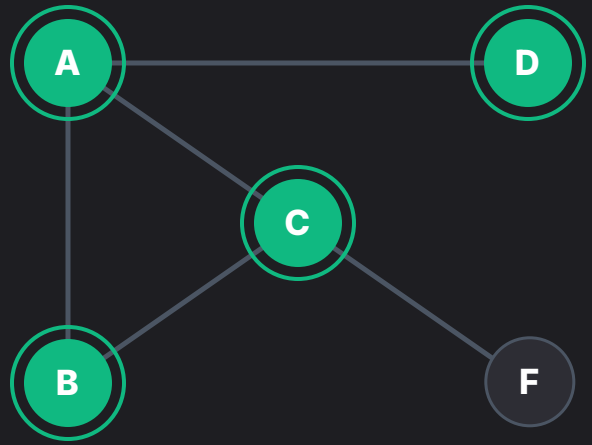
Rule: Explore children alphabetically when possible.

Visited

A	B	C	D	

Stack

F				



Traverse This Graph Using BFS

BFS Problem Setup

Suppose the goal is D. Which nodes will BFS visit before finding D?

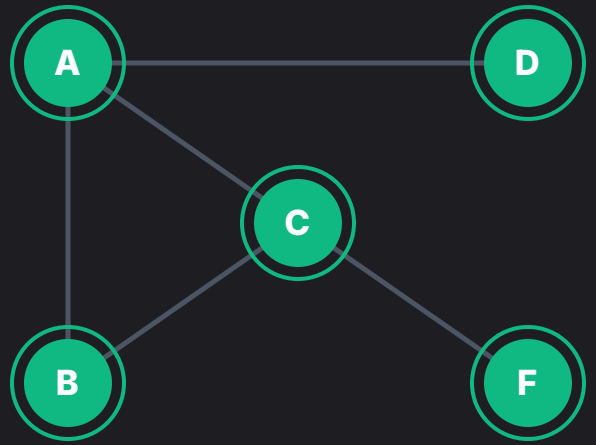
Rule: Explore children alphabetically when possible.

Visited

A	B	C	D	F

Stack

Why did we add the visited set?
The search can loop forever because it keeps rediscovering states.



Traverse This Graph Using BFS

In this graph, there are no costs written on the edges, so BFS treats every move as having the same cost, usually cost 1. BFS does not actually calculate the total cost of a path; instead, it explores the graph level by level, which means it first checks all nodes that are one step away from the start, then all nodes that are two steps away, and so on. Because every step has the same cost, a path with fewer steps also has a lower total cost. For example, from A to D there is only one edge, so the path has one step and cost 1. A longer path with three edges would have cost 3. Therefore, in an unweighted graph like this one, BFS finds the shortest path because it reaches nodes using the fewest number of steps first.

Pac-Man: BFS Pathfinding

Question & Setup

If Pac-Man uses DFS, will it necessarily find the shortest route to G?

P= Pac-Man

G= Goal

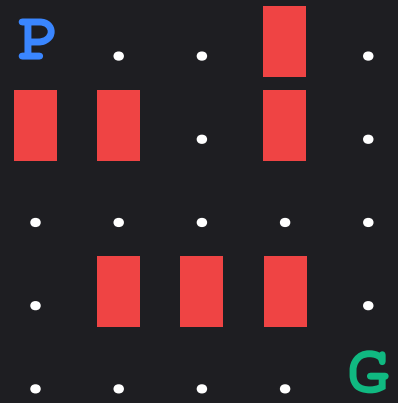
= Wall

Answer: Yes..IF

every move has the same cost, BFS will find the path with the fewest moves

BFS explores outward from Pac-Man like a wave until it reaches the goal.

Grid Environment Map



Pac-Man: BFS Pathfinding

Question & Setup

If Pac-Man uses DFS, will it necessarily find the shortest route to G?

P= Pac-Man

G= Goal

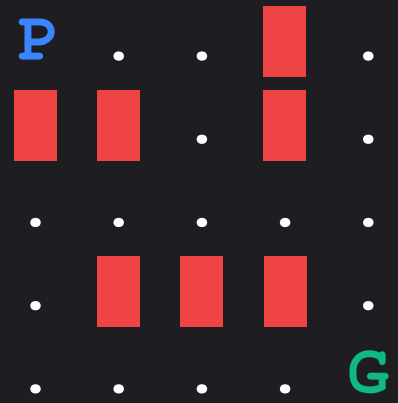
= Wall

Answer: Yes..IF

every move has the same cost, BFS will find the path with the fewest moves

BFS explores outward from Pac-Man like a wave until it reaches the goal.

Grid Environment Map



ANIMATED + CLICK-THROUGH

BFS vs DFS

Tree Search Traversal

BFS

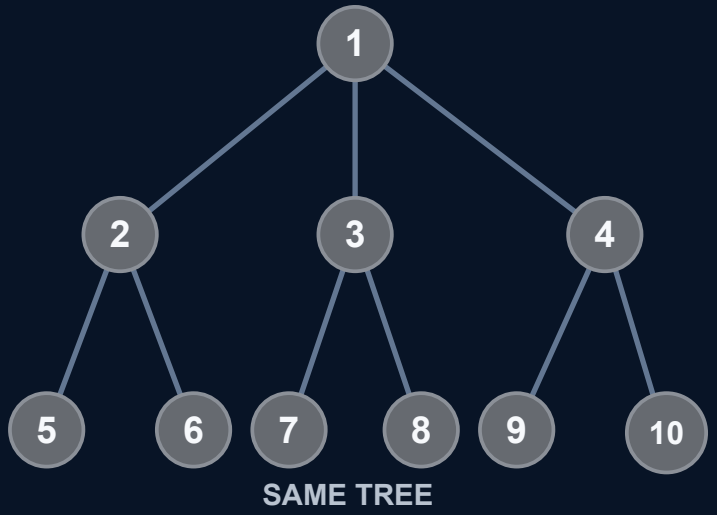
QUEUE / FIFO

Level by level

Visit order

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

First In, First Out



SAME TREE

Different data structures produce different traversal orders.

DFS

STACK / LIFO

Go deep, then backtrack

Visit order

1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

Last In, First Out

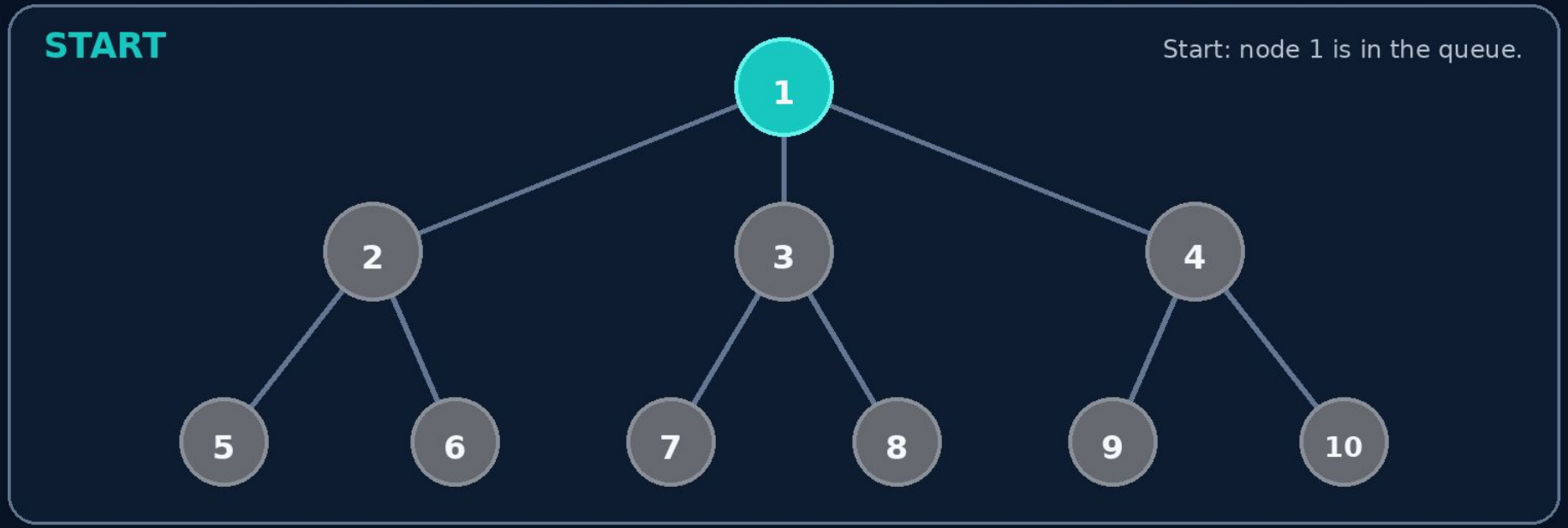
Yellow = current visit

Cyan = current queue/stack

Gray = inactive now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10



CURRENT QUEUE (FRONT → REAR)



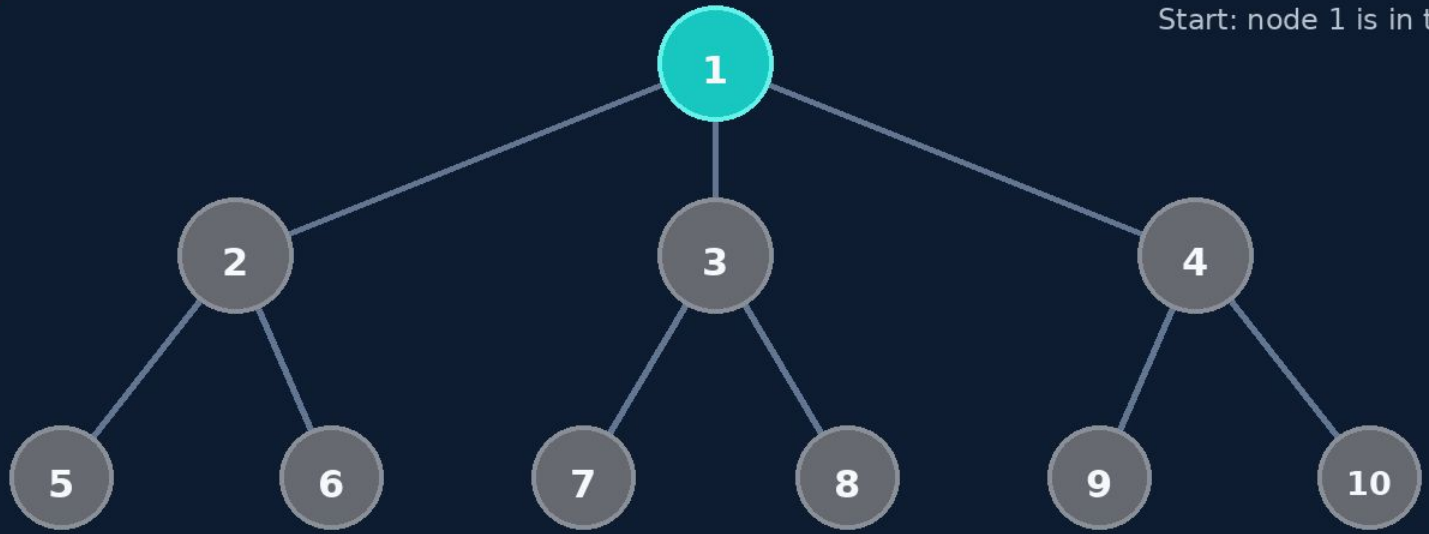
current visit currently in queue not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

START

Start: node 1 is in the queue.



CURRENT QUEUE (FRONT → REAR)



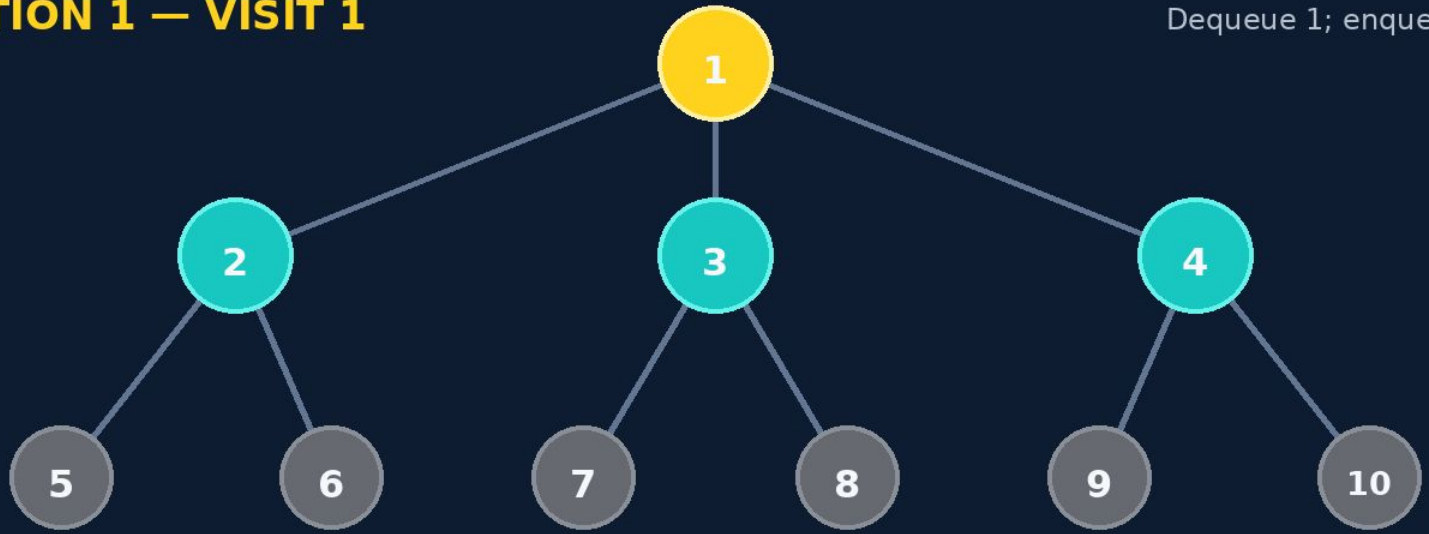
current visit currently in queue/stack not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

ITERATION 1 — VISIT 1

Dequeue 1; enqueue 2, 3, 4.



CURRENT QUEUE (FRONT → REAR)



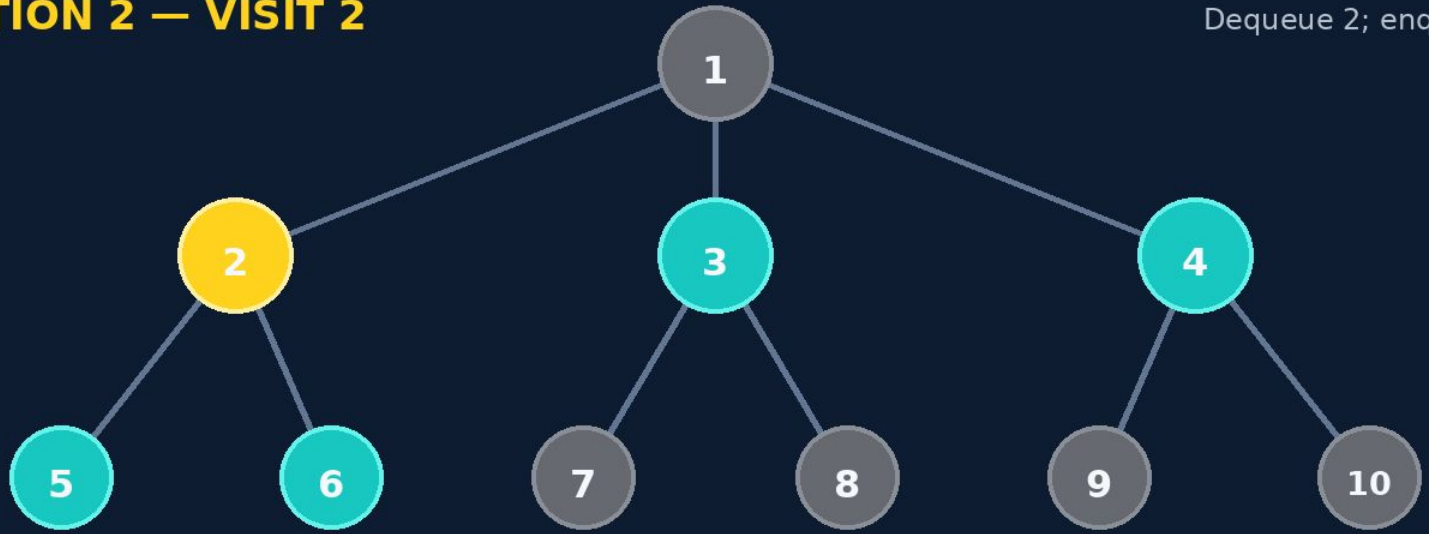
current visit · currently in queue/stack · not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

ITERATION 2 — VISIT 2

Dequeue 2; enqueue 5, 6.



CURRENT QUEUE (FRONT → REAR)



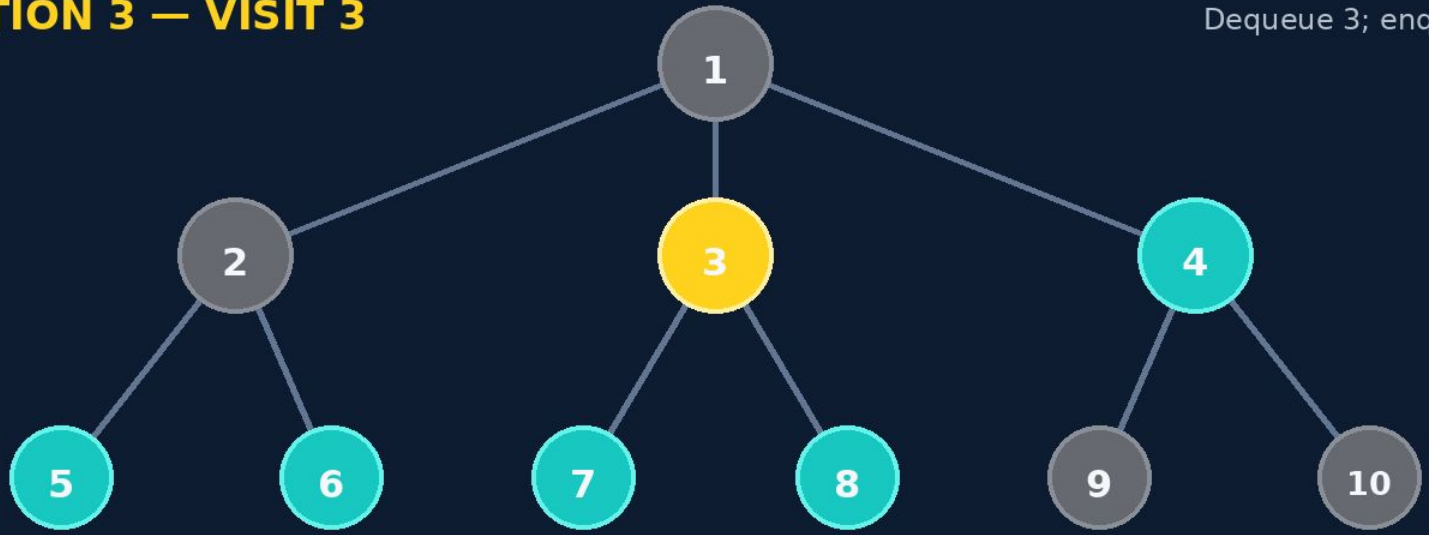
current visit · currently in queue/stack · not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

ITERATION 3 — VISIT 3

Dequeue 3; enqueue 7, 8.



CURRENT QUEUE (FRONT → REAR)



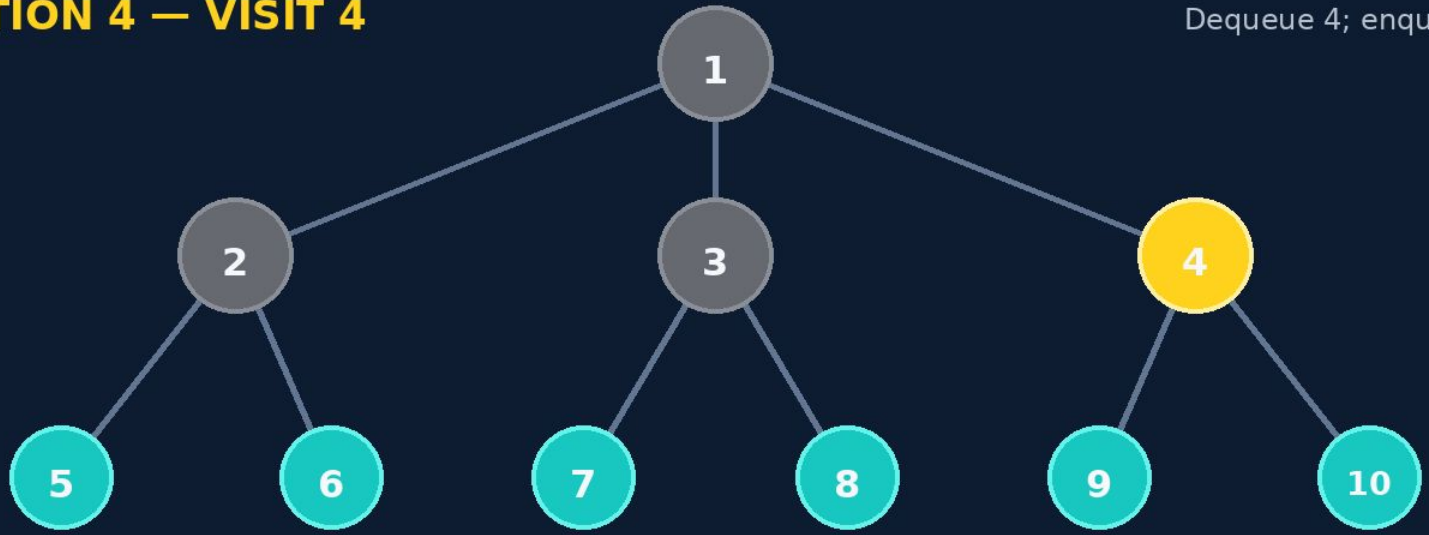
current visit · currently in queue/stack · not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

ITERATION 4 — VISIT 4

Dequeue 4; enqueue 9, 10.



CURRENT QUEUE (FRONT → REAR)



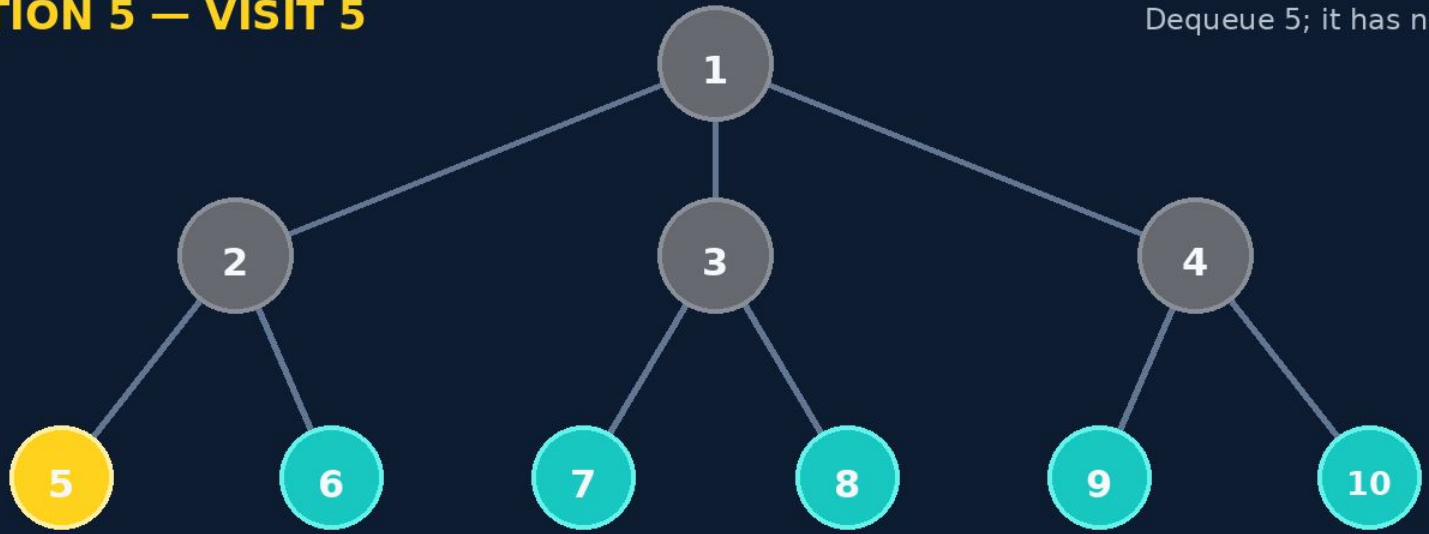
current visit currently in queue/stack not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

ITERATION 5 — VISIT 5

Dequeue 5; it has no children.



CURRENT QUEUE (FRONT → REAR)



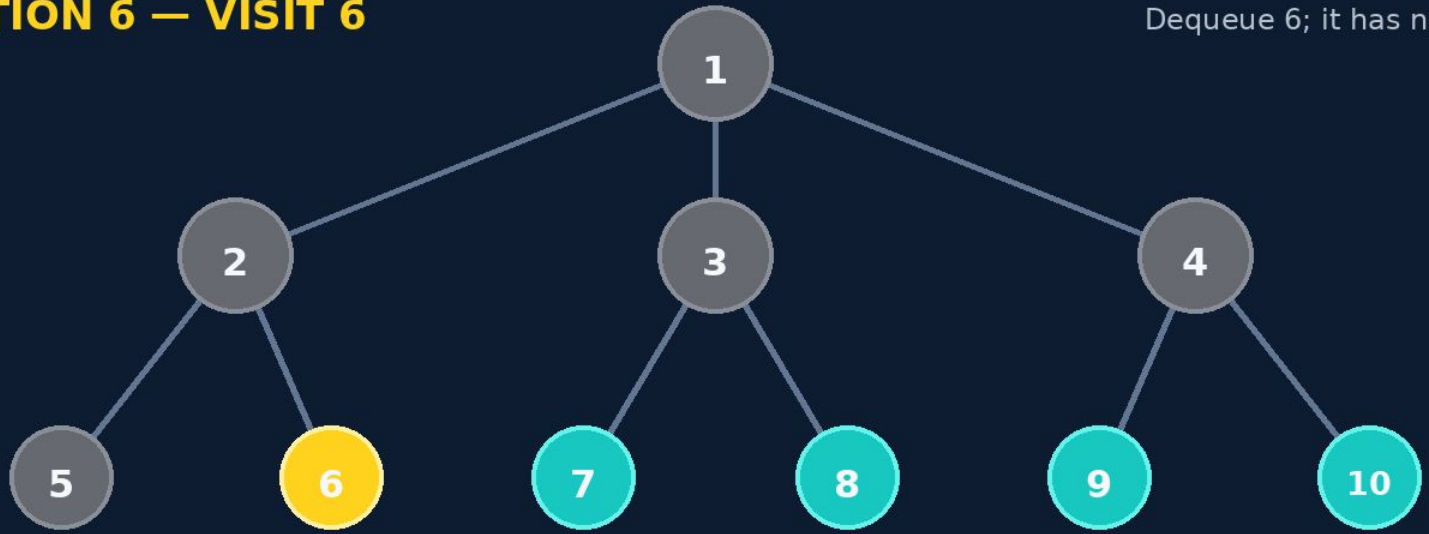
current visit · currently in queue/stack · not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

ITERATION 6 — VISIT 6

Dequeue 6; it has no children.



CURRENT QUEUE (FRONT → REAR)



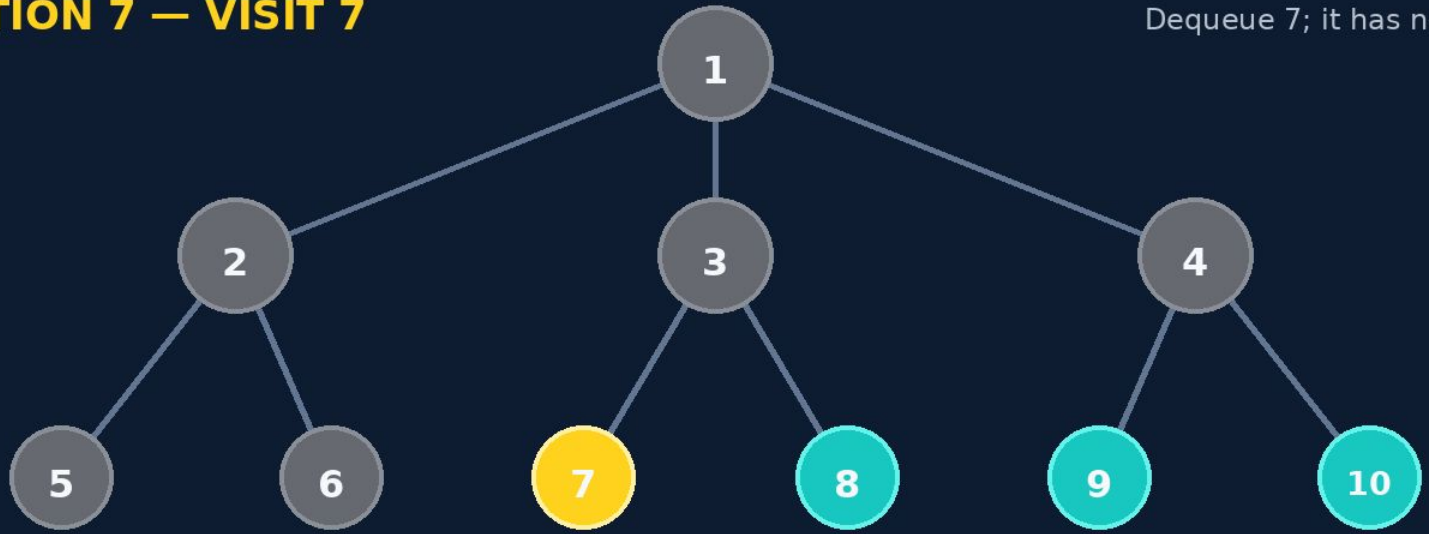
current visit · currently in queue/stack · not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

ITERATION 7 — VISIT 7

Dequeue 7; it has no children.



CURRENT QUEUE (FRONT → REAR)



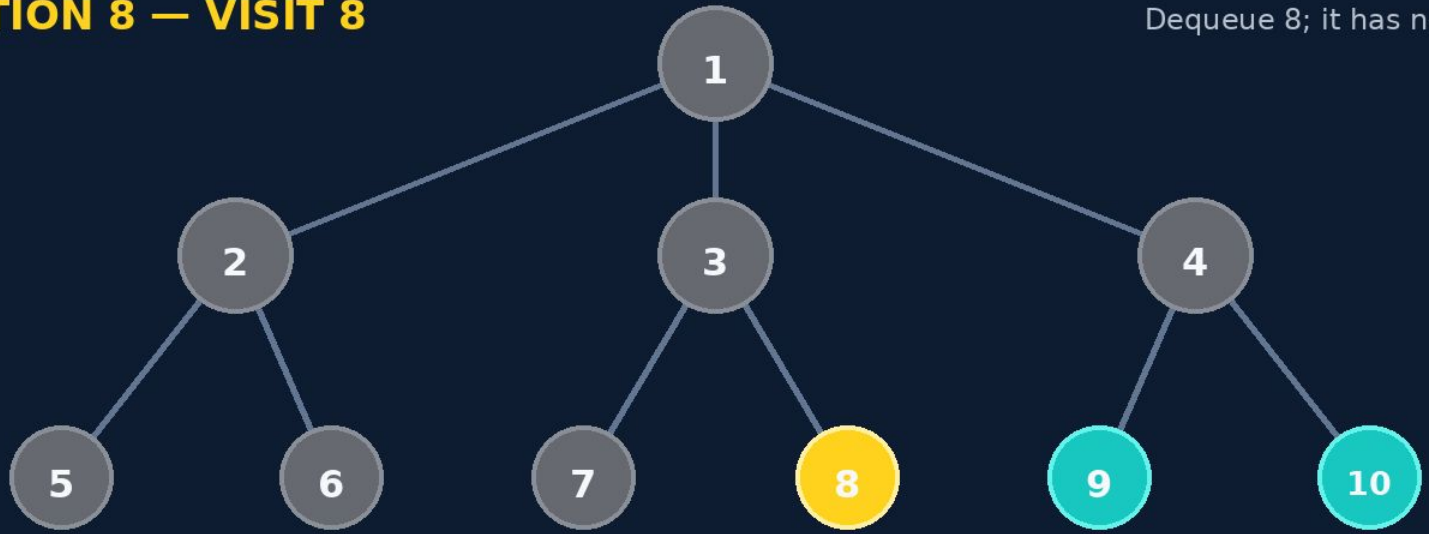
current visit · currently in queue/stack · not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

ITERATION 8 — VISIT 8

Dequeue 8; it has no children.



CURRENT QUEUE (FRONT → REAR)



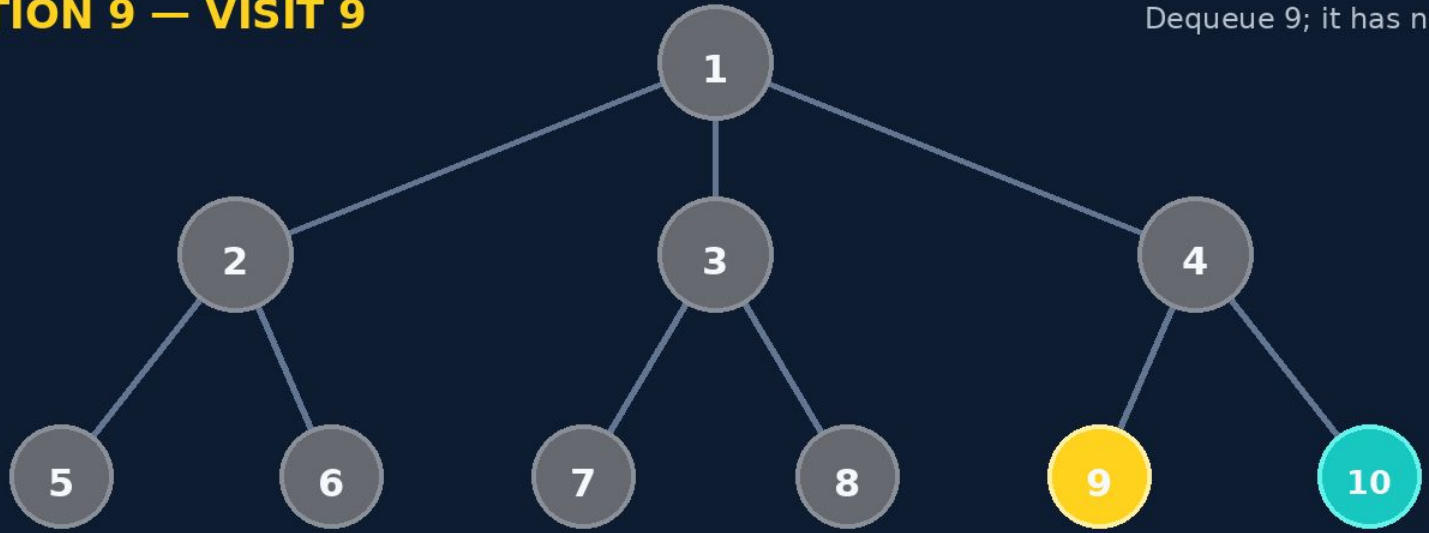
current visit · currently in queue/stack · not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

ITERATION 9 — VISIT 9

Dequeue 9; it has no children.



CURRENT QUEUE (FRONT → REAR)



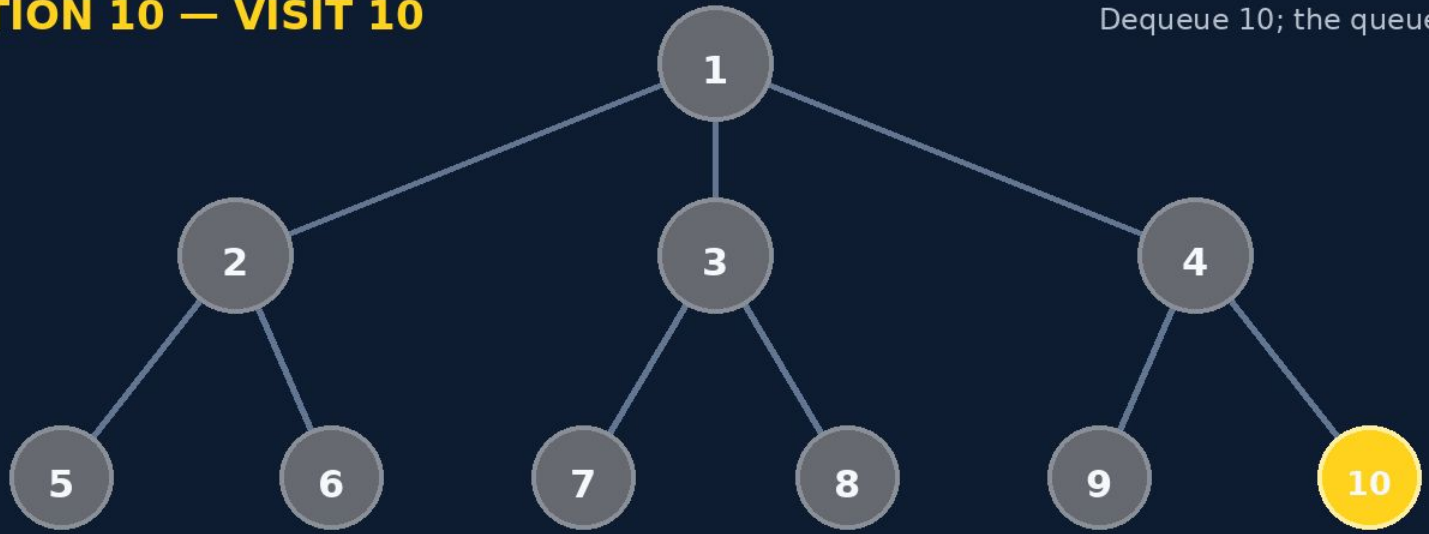
current visit currently in queue/stack not active now

BFS ANIMATION — QUEUE / FIFO

Visit order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

ITERATION 10 — VISIT 10

Dequeue 10; the queue is empty.



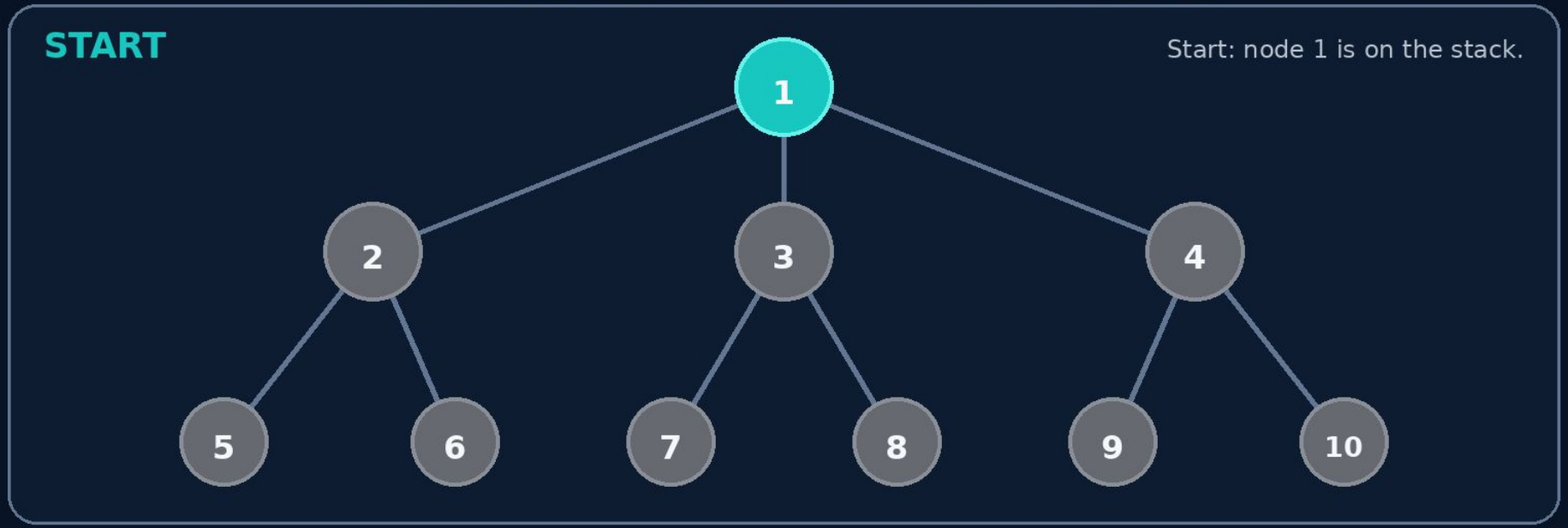
CURRENT QUEUE (FRONT → REAR)

[EMPTY]

current visit | currently in queue/stack | not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10



CURRENT STACK (TOP → BOTTOM)



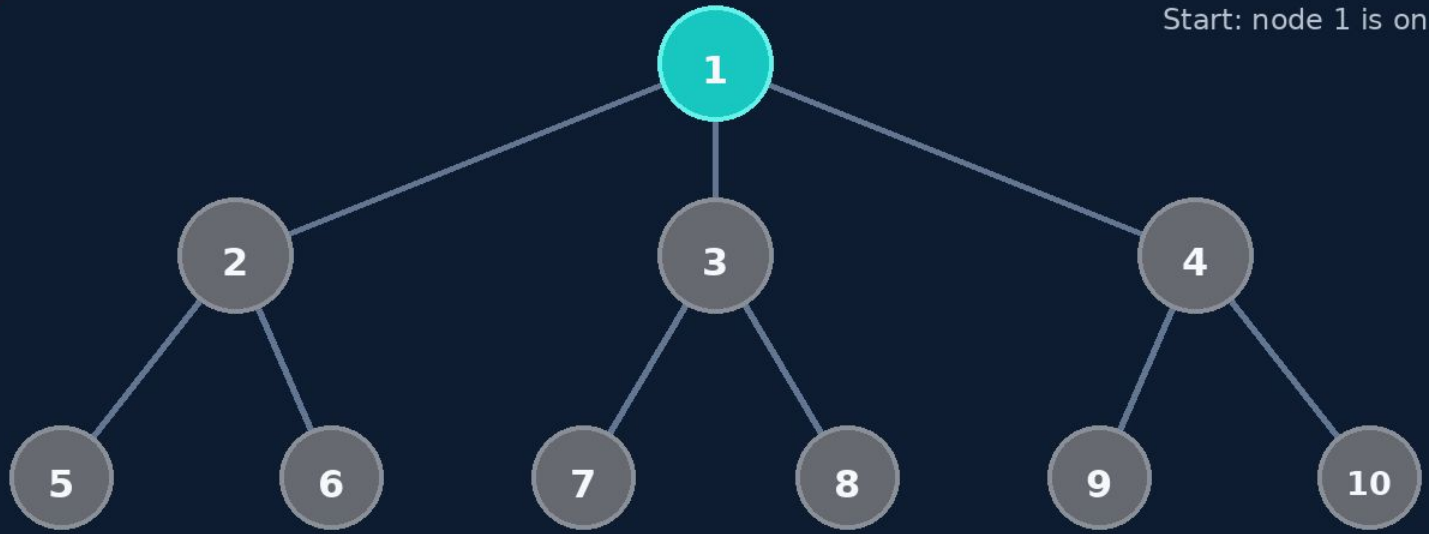
current visit currently in stack not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

START

Start: node 1 is on the stack.



CURRENT STACK (TOP → BOTTOM)



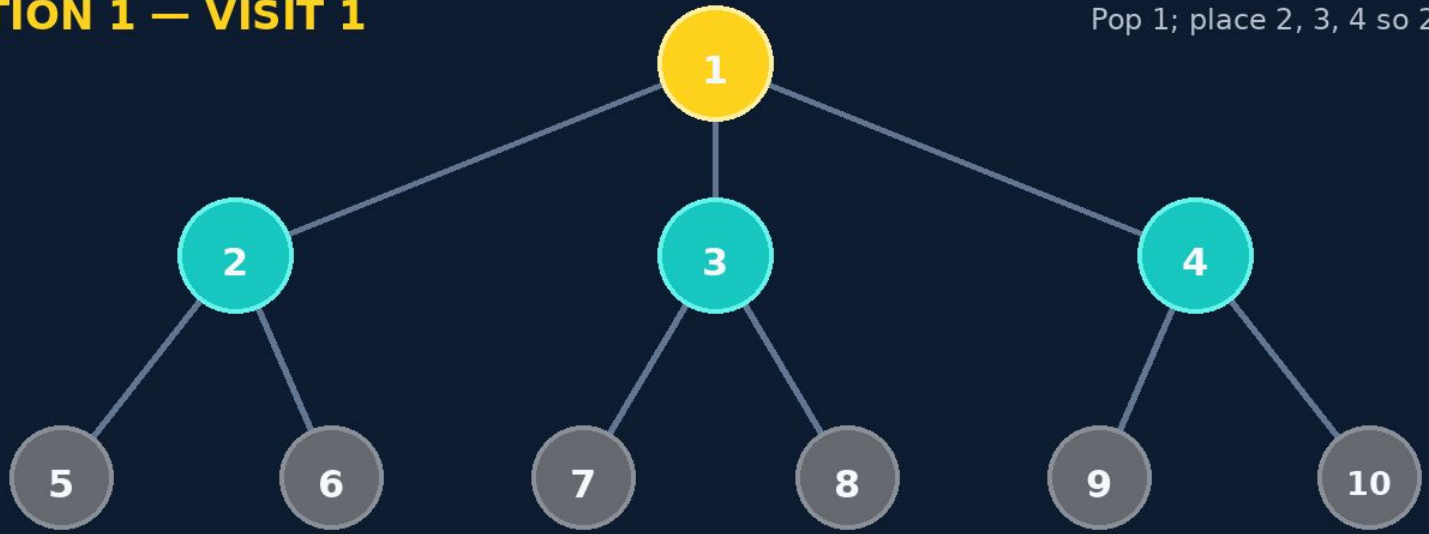
current visit currently in queue/stack not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

ITERATION 1 — VISIT 1

Pop 1; place 2, 3, 4 so 2 is on top.



CURRENT STACK (TOP → BOTTOM)



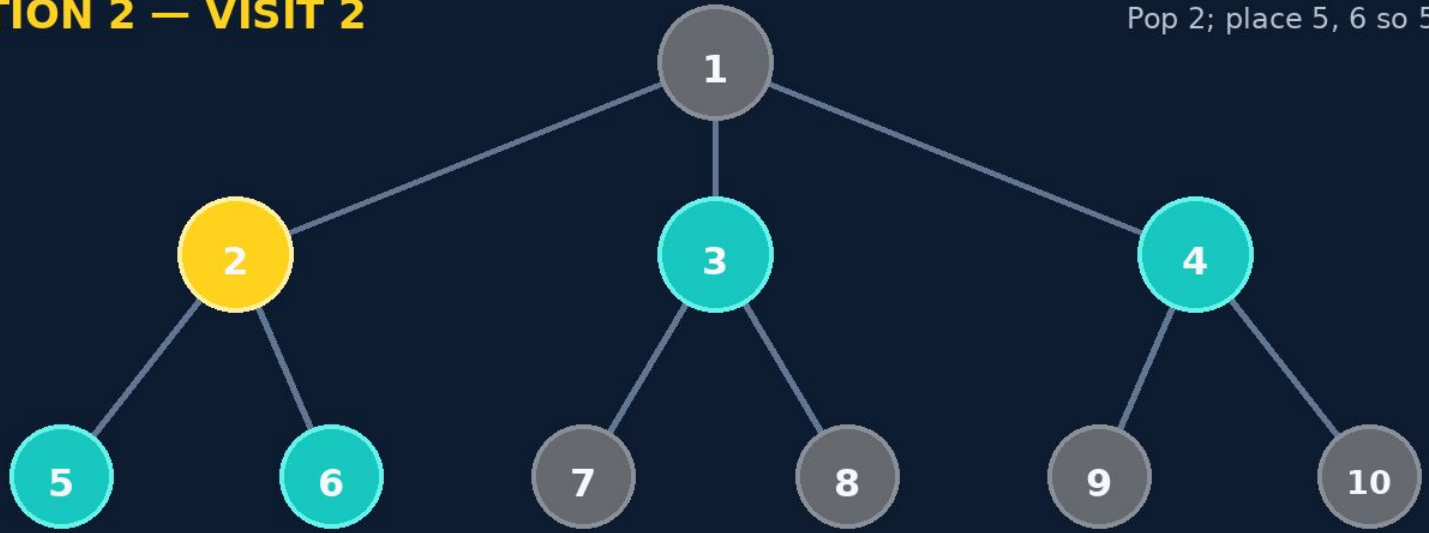
current visit currently in queue/stack not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

ITERATION 2 — VISIT 2

Pop 2; place 5, 6 so 5 is on top.



CURRENT STACK (TOP → BOTTOM)



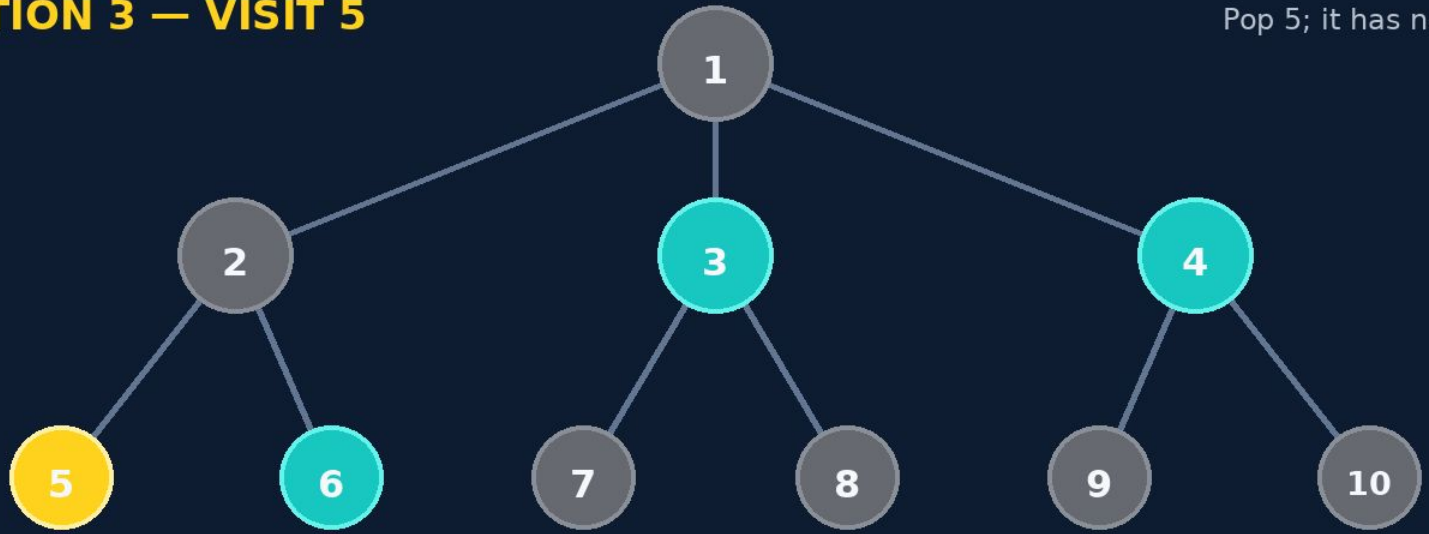
current visit · currently in queue/stack · not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

ITERATION 3 — VISIT 5

Pop 5; it has no children.



CURRENT STACK (TOP → BOTTOM)



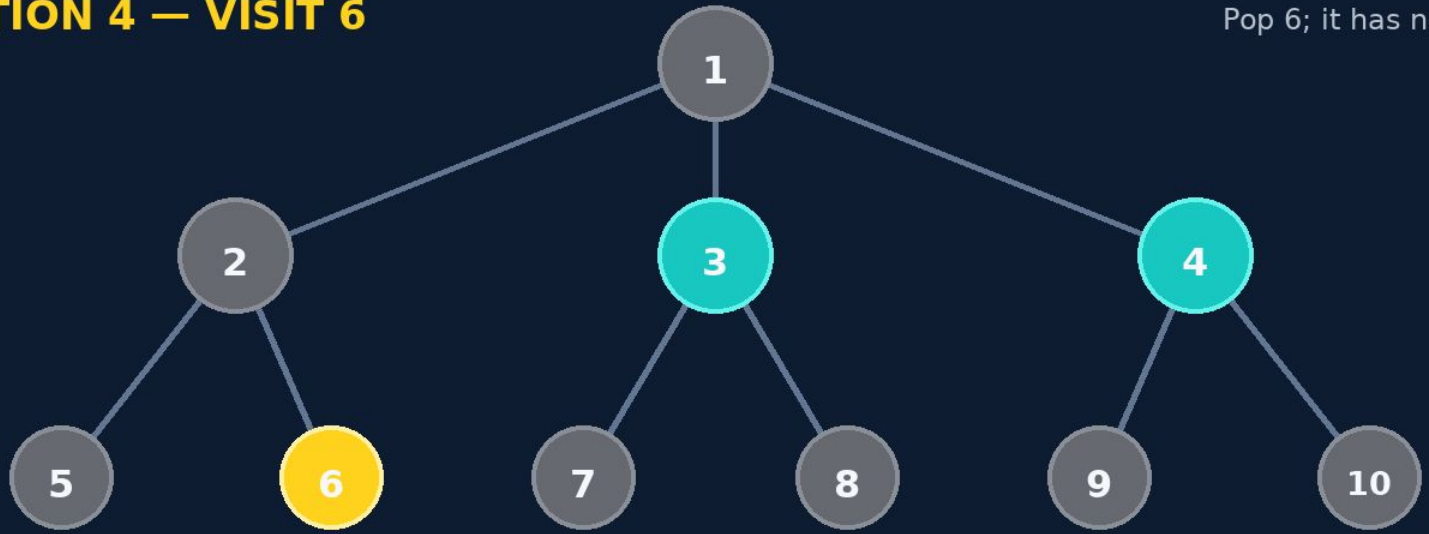
current visit · currently in queue/stack · not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

ITERATION 4 — VISIT 6

Pop 6; it has no children.



CURRENT STACK (TOP → BOTTOM)



TOP

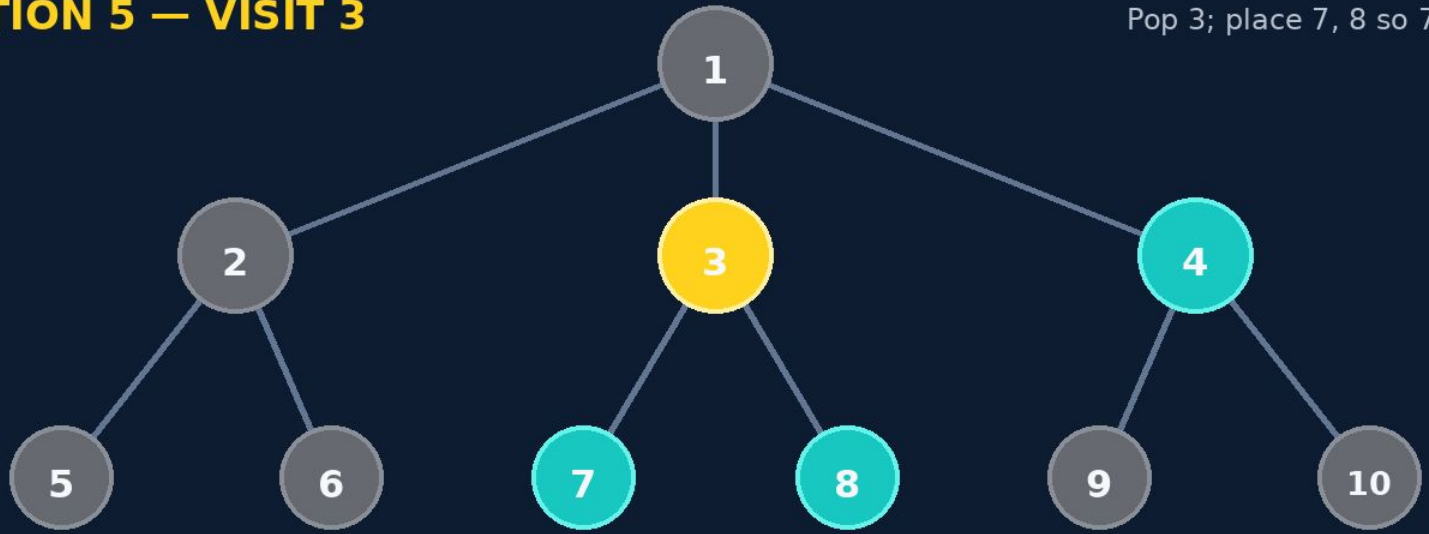
current visit currently in queue/stack not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

ITERATION 5 — VISIT 3

Pop 3; place 7, 8 so 7 is on top.



CURRENT STACK (TOP → BOTTOM)



TOP

current visit

currently in queue/stack

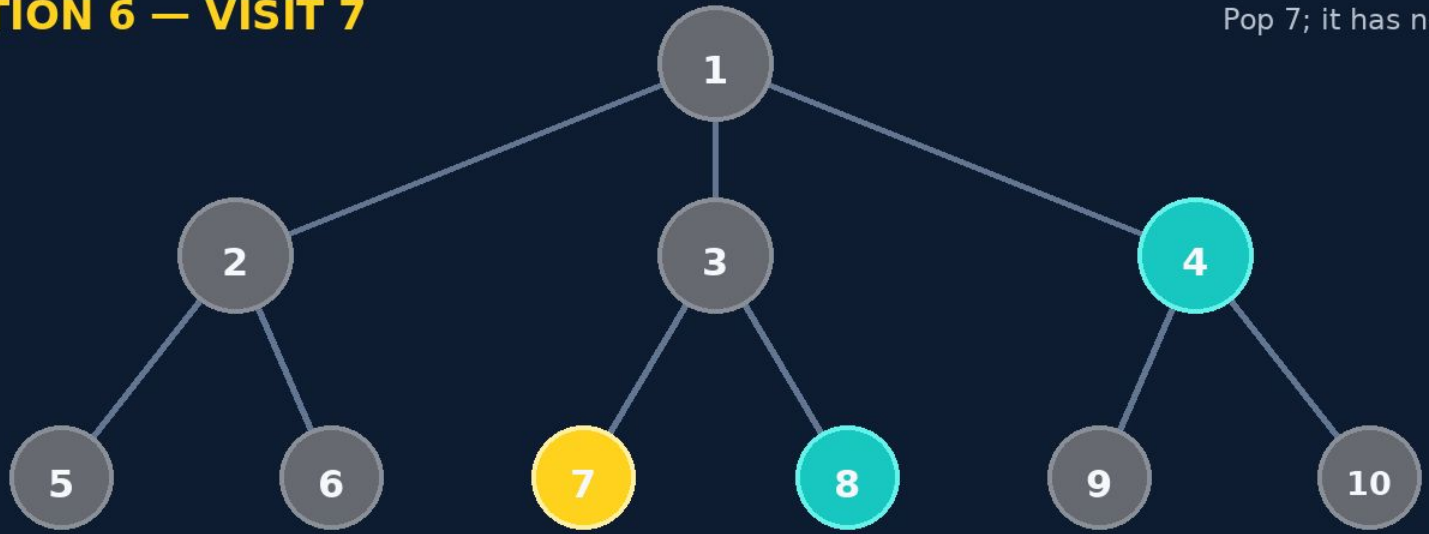
not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

ITERATION 6 — VISIT 7

Pop 7; it has no children.



CURRENT STACK (TOP → BOTTOM)



TOP

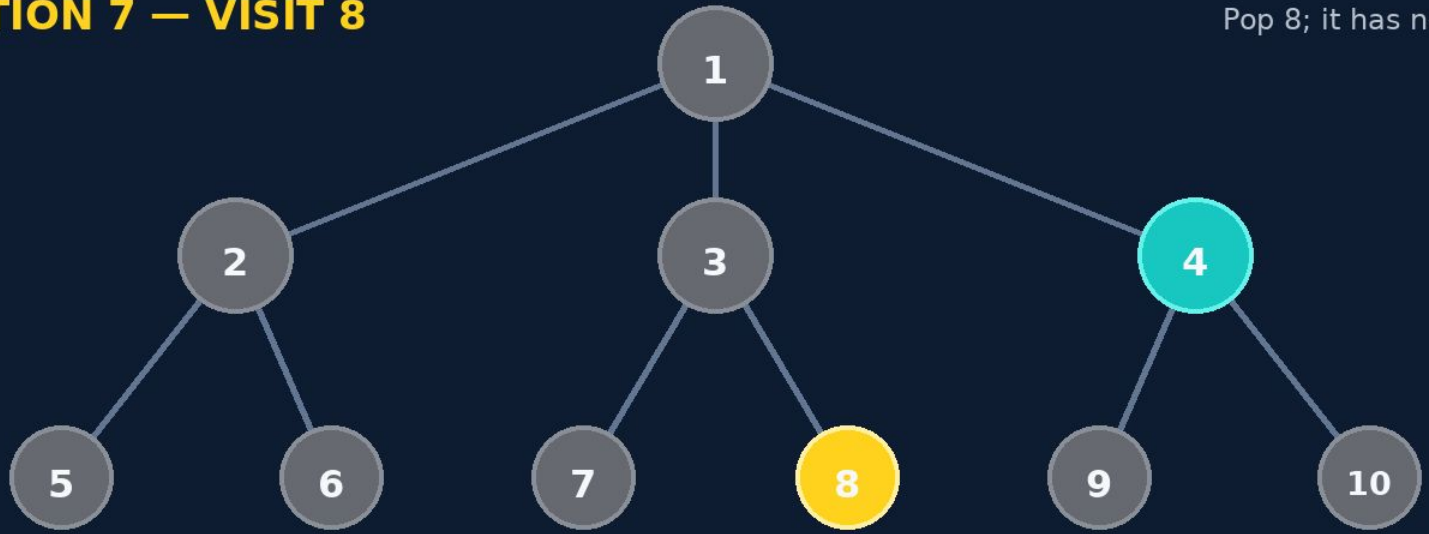
current visit currently in queue/stack not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

ITERATION 7 — VISIT 8

Pop 8; it has no children.



CURRENT STACK (TOP → BOTTOM)



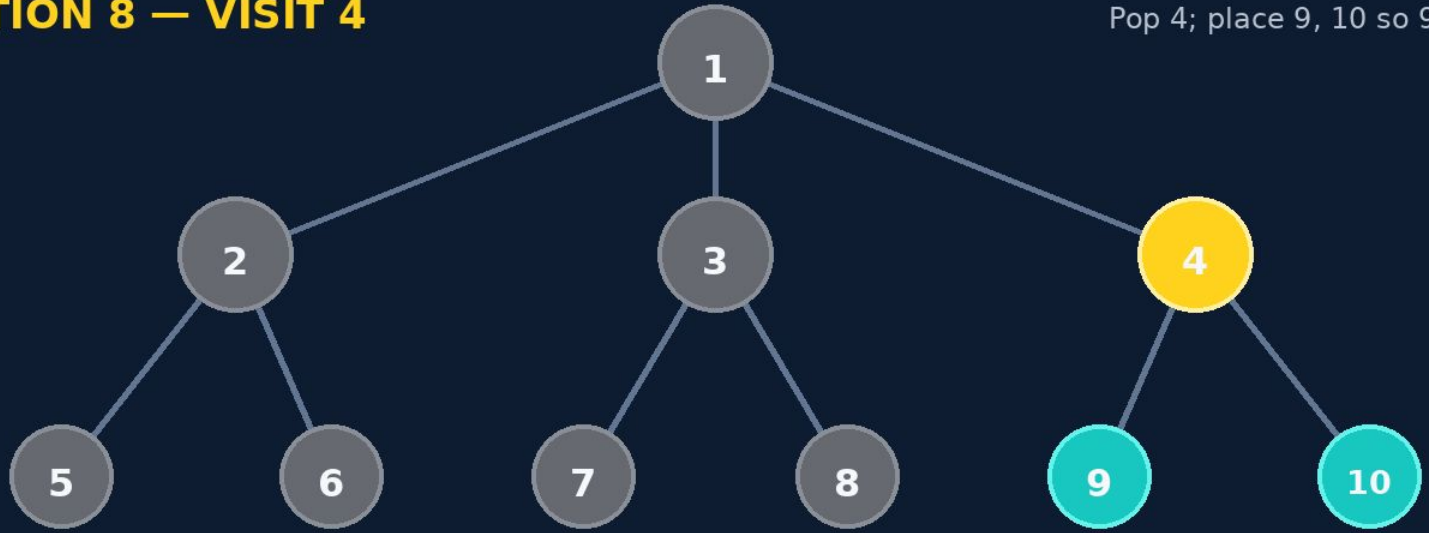
current visit currently in queue/stack not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

ITERATION 8 — VISIT 4

Pop 4; place 9, 10 so 9 is on top.



CURRENT STACK (TOP → BOTTOM)



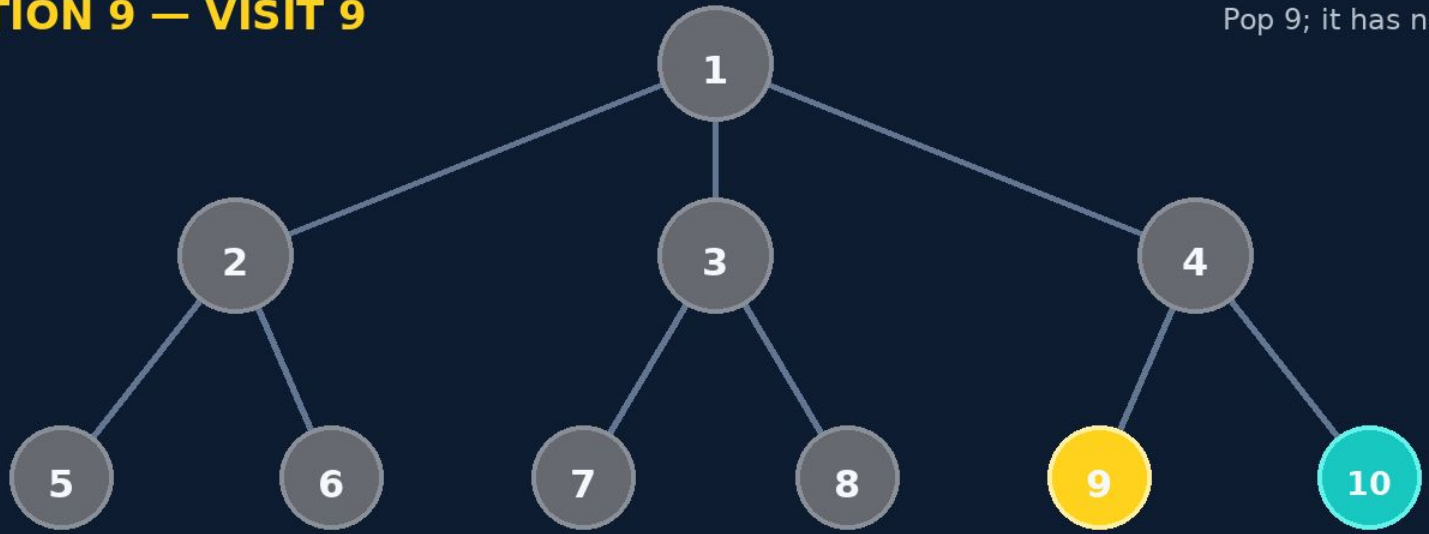
current visit · currently in queue/stack · not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

ITERATION 9 — VISIT 9

Pop 9; it has no children.



CURRENT STACK (TOP → BOTTOM)



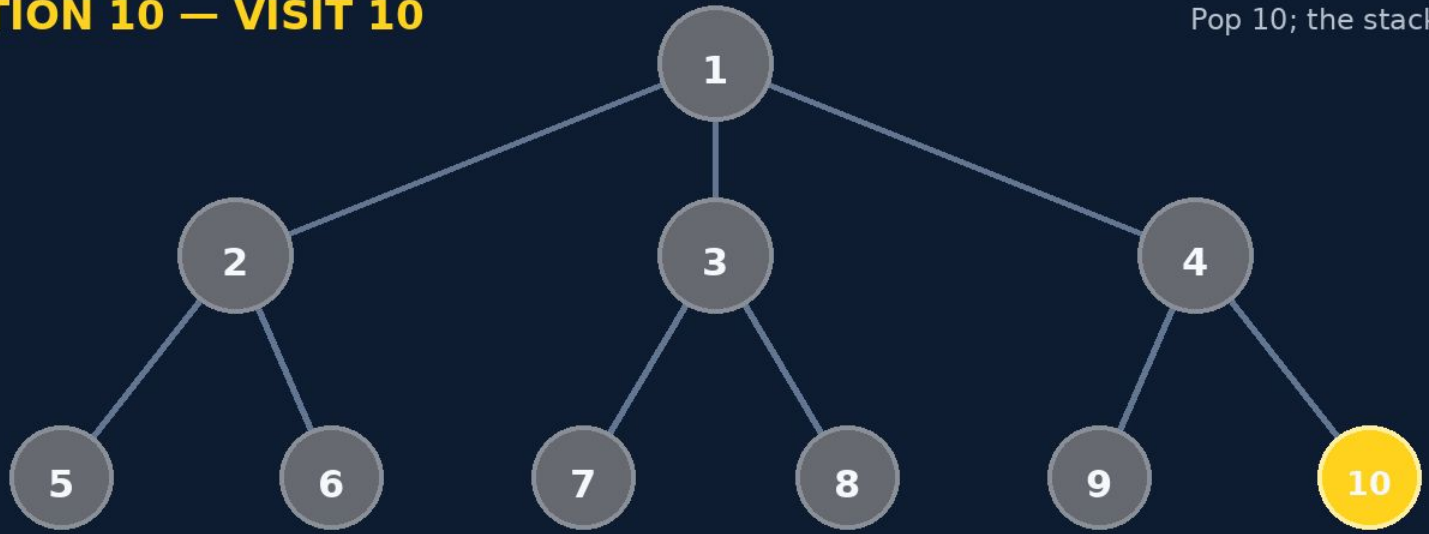
current visit currently in queue/stack not active now

DFS ANIMATION — STACK / LIFO

Visit order: 1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

ITERATION 10 — VISIT 10

Pop 10; the stack is empty.



CURRENT STACK (TOP → BOTTOM)

[EMPTY]

current visit | currently in queue/stack | not active now

BFS vs DFS — Final Comparison

Same tree. Different frontier data structure. Different visit order.

BREADTH-FIRST SEARCH

Queue • FIFO

Visit order

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

Behavior

Finishes the current depth level before moving deeper.

Next after visiting 2: 3

DEPTH-FIRST SEARCH

Stack • LIFO

Visit order

1 → 2 → 5 → 6 → 3 → 7 → 8 → 4 → 9 → 10

Behavior

Follows one branch as deeply as possible, then backtracks.

Next after visiting 2: 5

Yellow = current visit

Cyan = current queue/stack

Gray = inactive now

BFS: Strengths and Weaknesses

STRENGTHS

- **Complete** if the state space is finite.
- Finds the **shortest path** (fewest actions) when all actions have **equal cost**.
- Highly effective for shortest-path problems in **unweighted graphs**.

WEAKNESSES

- Can use a **lot of memory** (exponential space complexity).
- May explore **many irrelevant nodes** before reaching a deep goal.

Why BFS Is Not Enough

BFS may find the path with the fewest steps, but the fewest steps may not be the cheapest path when costs differ.

PATH A FEWER STEPS

2 Roads (Fewer actions)

Total Cost = 20 (Expensive)

BFS may prefer this path simply because it has fewer steps, ignoring the high cumulative cost.

PATH B OPTIMAL CHOICE

4 Roads (More actions)

Total Cost = 8 (Much cheaper!)

If we care about total cost, Path B is better. BFS fails to find this optimal solution because of step-cost blindness.

This fundamental limitation motivates **Uniform-Cost Search (UCS)**, which evaluates path weights to find the true cheapest route.

Uniform-Cost Search

CORE STRATEGY

Lowest-Cost Expansion

Uniform-Cost Search (UCS) systematically explores the state space by expanding the frontier node with the **lowest total path cost** accumulated so far.

To manage this order efficiently, UCS maintains its frontier using a **priority queue**.

MATHEMATICAL METRIC

Path Cost Function

$$g(n)$$

Where ***g(n)*** represents the exact cumulative cost of the path from the **start node** to node ***n***.

At each step, UCS always asks: ***"Which partial path is cheapest so far?"***

UCS Weighted Example

STEP 1 — START AT A

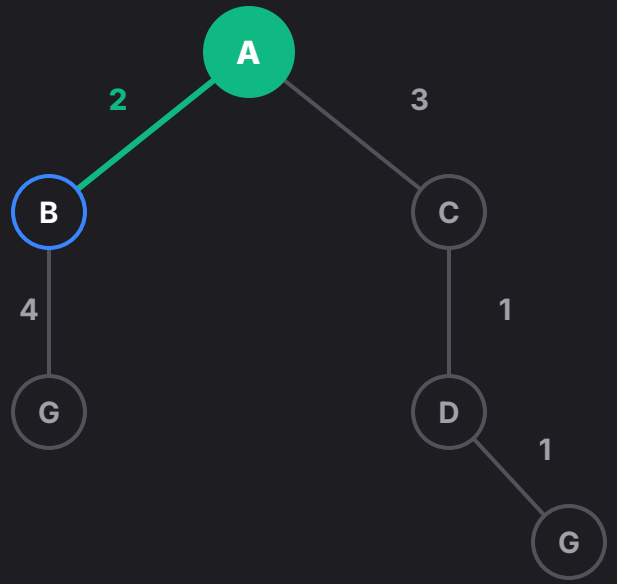
Start: A(0)

Expand **A**. We discover two paths:

- **B: 0 + 2 = 2**
- **C: 0 + 3 = 3**

Frontier: **B(2), C(3)**

UCS selects the lowest-cost node:
Expand B(2)



UCS Weighted Example

STEP 2 — EXPAND B

Path: A→B (Cost: 2)

From B we can reach G: **g(G) = 2 + 4 = 6**

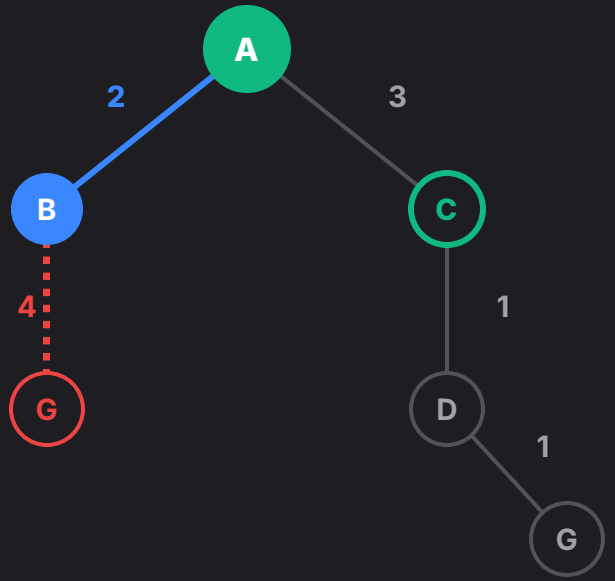
Frontier: **C(3), G(6)**

QUESTION: Should UCS stop now?

No. C(3) in the frontier has a lower cost than the goal G(6).

"We must expand C because a cheaper path to goal might exist through it."

UCS selects: Expand C(3)



UCS Weighted Example

STEP 3 — EXPAND C

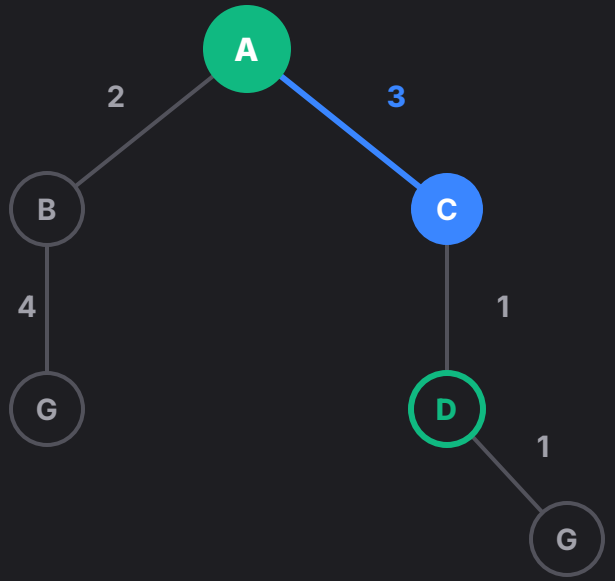
Path: A→C (Cost: 3)

From C we can reach D: **g(D) = 3 + 1 = 4**

Frontier: **D(4), G(6)**

UCS selects the lowest-cost node in the frontier:

Expand D(4)



UCS Weighted Example

STEP 4 — EXPAND D

Path: A→C→D (Cost: 4)

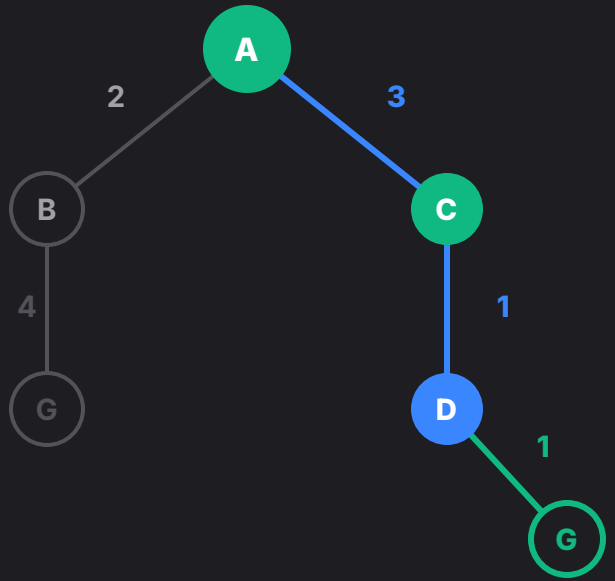
From D we can reach G: **g(G) = 4 + 1 = 5**

Previous cost to G: **G(6) via B**

CHEAPER ROUTE FOUND!

Frontier updated: **G(6) ➔ G(5)**

"The old cost-6 path is replaced by the better cost-5 path."



UCS Weighted Example

STEP 5 — SELECT G

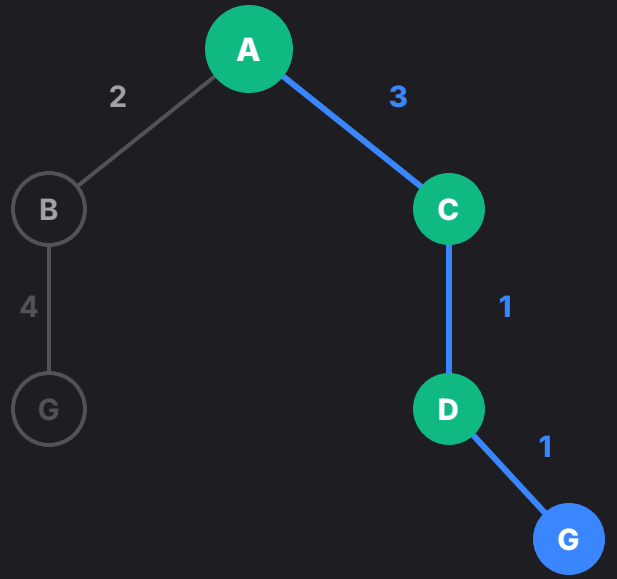
Path: A→C→D→G

Lowest-cost frontier node is: **G(5)**

UCS removes G and stops.

Optimal Solution Reached!
Total Cost: 3 + 1 + 1 = **5**

UCS does not stop when the goal is discovered. It stops when the goal is removed from the frontier as the lowest-cost node.



Fun Fact

Uniform-Cost Search is essentially **Dijkstra's shortest-path algorithm** used in an AI search setting.

UCS Real Example: Movement Costs

In pathfinding, different terrain can have different movement costs.

Grass

Movement Cost:
Low Cost

Forest

Movement Cost:
Higher Cost

Water

Movement Cost:
Very High Cost

A path with fewer steps may not be the cheapest path.

UCS: Strengths and Weaknesses

Strengths

- Find the lowest-cost solution when costs are nonnegative
- Handles weighted paths better than BFS
- Useful for route planning and cost-sensitive

Weaknesses

- Can explore many low-cost paths that do not move toward the goal
- Needs a priority queue
- Can use a lot of memory

Algorithm	Frontier	Expands	Complete?	Optimal?
DFS	Stack	Deepest node	Not always	No
BFS	Queue	Shallowest node	Yes	Only with equal costs
UCS	Priority queue	Lowest-cost node	Yes	Yes, with nonnegative costs

Which Algorithm would you choose?

Which Algorithm would you choose?

Scenario 1

You are solving a maze where every move has the same cost, and you want the fewest moves

Which Algorithm would you choose?

Scenario 2

You are navigating a map where roads have different travel times.

Which Algorithm would you choose?

Scenario 3

You want a simple algorithm that may quickly find a solution, but the solution does not need to be shortest.

DFS, BFS, and UCS are useful, but they search without knowing where the goal is.

Next, we ask:

“Can the agent use an estimate of how close it is to the goal?”

Search Algorithms Are Used in Real Products

The textbook names may change, but the same graph-search ideas appear in commercial platforms.

Lecture	Company / Product	What the product calls it	Same idea from class
BFS	**Amazon Neptune Analytics**	Breadth-first graph traversal	Shallowest levels first
DFS	**Neo4j Graph Data Science**	Depth-first graph traversal	Deep branch before backtracking
UCS	**Neo4j — Dijkstra**	Weighted shortest path	Lowest accumulated cost g(n)
A*	**Neo4j Graph Data Science**	Heuristic shortest path	g(n) + h(n)

INDUSTRY LESSON **UCS is often encountered in industry as Dijkstra; A* adds a heuristic to focus the search.**

Sources: AWS Neptune Analytics algorithms; Neo4j Graph Data Science path-finding documentation (official product docs).

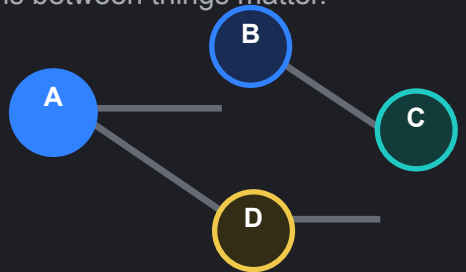
First: What Is Amazon Neptune Analytics?

An AWS service built to analyze highly connected data as a graph — nodes connected by relationships.

AWS SERVICE

Think in relationships

Traditional tables are great for rows. Neptune Analytics is designed for questions where the connections between things matter.



Nodes	people • devices • airports • accounts
Edges	friends • routes • transfers • links

How large can the analysis be?

TENS OF BILLIONS

of relationships in a graph

AWS states that Neptune Analytics can load large graph datasets into memory and analyze graphs with tens of billions of relationships, while processing thousands of analytic queries per second.

Example workloads

RECOMMENDATIONS | FRAUD | NETWORK THREATS

The key idea: instead of scanning isolated records, graph algorithms follow relationships to reveal patterns.

Official source: AWS Neptune Analytics User Guide — “When to use Neptune Analytics and when to use Neptune Database.”

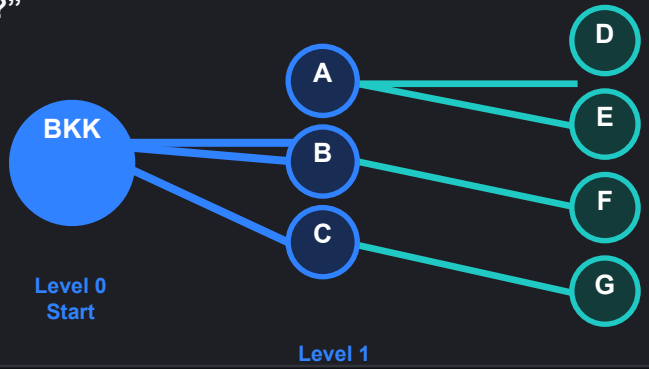
Real AWS Example: BFS Through an Airport Route Graph

AWS documentation demonstrates BFS on its sample air-routes graph, including searches starting from Bangkok (BKK).

AIR-ROUTES

Question for BFS

“Starting at BKK, which airports can I reach in 1 hop, then 2 hops?”




```
MATCH (n {code:"BKK"}) CALL
neptune.algo.bfs.levels(n,{edgeLabels:["route"],maxDepth:2})
```

How BFS helps

1. Start from one airport.
2. Visit all airports one route away.
3. Then visit airports two routes away.
4. Continue level by level until the target is found.

With equal edge cost, the first time BFS reaches a destination corresponds to the fewest route hops.

Important: this means fewest connections — not necessarily cheapest fare or shortest travel time.

Official source: AWS Neptune Analytics BFS documentation and sample air-routes examples (BKK / route edges). Query shown is adapted to the documented syntax for teaching.

Amazon Neptune Analytics: BFS in a Real Product

AWS exposes Breadth-First Search directly as a graph algorithm.

AWS NEPTUNE

What AWS documents

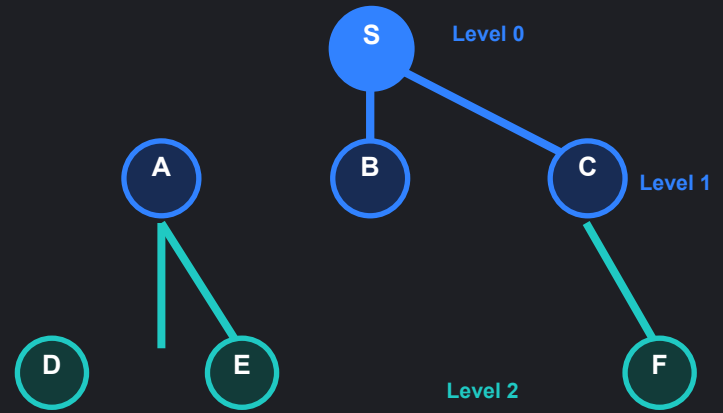
- Explore all neighboring nodes before moving to the next level.
- Searches the shallowest levels of a graph first.
- AWS lists networking, social networks, and games as BFS use cases.

Example API

```
CALL neptune.algo.bfs(source, {...})
```

BFS expands level by level

Example: find devices with the fewest network hops



Class connection: with equal edge costs, BFS reaches nodes by the fewest hops first.

Official source: AWS Neptune Analytics User Guide — Breadth-first search (BFS) path-finding algorithms.

Neo4j: From DFS to Dijkstra (UCS) to A*

One commercial graph platform exposes several algorithms from the same search family.

TRAVERSAL

DFS

Go deep first

Explores as far as possible along a branch before backtracking.

```
Stack / depth-first behavior
```

UCS IN INDUSTRY

Dijkstra

Cheapest path so far

Supports weighted graphs with positive relationship weights.

```
Priority = g(n)
```

INFORMED SEARCH

A*

Cheapest estimated total

Neo4j combines computed distance with a heuristic; its GDS heuristic uses Haversine distance.

```
Priority = g(n) + h(n)
```

KEY TAKEAWAY

The algorithms in the lecture are not historical curiosities — they are exposed as production graph algorithms today.

REAL PRODUCTS

Official sources: Neo4j Graph Data Science — Path finding, DFS, Dijkstra Source-Target, and A* Shortest Path documentation.